Lecture-#11

Natural Language Processing

A8706

UNIT-III

Semantic Parsing

Semantic Parsing:

❖Semantic Parsing is the task of transducing natural language utterances into formal meaning representations.

❖The target meaning representations can be defined according to a wide variety of formalisms.

❖Semantic analysis analyzes the grammatical format of sentences, including the arrangement of words, phrases, and clauses, to determine relationships between independent terms in a specific context. This is a crucial task of natural language processing (NLP) systems.

Why is semantic parsing important?

❖Semantic parsing has become important in facilitating human-system interaction due to its unique ability to convert natural language utterances into structured semantic representations.

❖These representations can then be executed against systems to obtain the desired responses for human users.

Semantic Parsing:

❖Semantic parsing is a subfield of natural language processing (NLP) that involves mapping natural language sentences or phrases into meaningful representations of their meaning, known as semantic representations. These representations can be used for various applications, such as:

Goals of Semantic Parsing:

- ❖ **Machine Comprehension:** Enable machines to understand the meaning of text.
- ❖ **Information Extraction:** Extract specific information from text.
- ❖ **Question Answering:** Answer questions based on the meaning of text.
- ❖ **Text Summarization:** Summarize text while preserving its meaning.
- ❖ **Dialogue Systems:** Improve human-machine conversation.

Semantic Parsing:

❖Key Components:

- ❖ **Syntax:** Analyze sentence structure.
- ❖ **Semantics:** Identify meaning of words, phrases, and relationships.
- ❖ **Pragmatics:** Consider context and inference.

❖Techniques:

- ❖ **Rule-based approaches:** Hand-crafted rules.
- ❖ **Machine learning-based approaches:** Train models on annotated data.
- ❖ **Hybrid approaches:** Combine rule-based and machine learning-based methods.

❖Applications:

- ❖ **Virtual Assistants:** Improve intent recognition.
- ❖ **Search Engines:** Enhance query understanding.
- ❖ **Chatbots:** Better comprehend user input.

Semantic Parsing:

How is semantic parsing done in NLP?

❖Semantic parsing is the task of translating natural language into a formal meaning representation on which a machine can act.

❖Representations may be an executable language such as SQL or more abstract representations such as Abstract Meaning Representation (AMR) and Universal Conceptual Cognitive Annotation (UCCA).

❖A domain-dependent, deeper representation and a set of relatively shallow but general-purpose, low-level, and intermediate representations. The task of producing the output of the first type is often called **deep semantic parsing**, and the task of producing the output of the second type is often called **shallow semantic parsing.**

Semantic Parsing:

❖**Example:**

Shallow semantic parsers can parse utterances like "show me flights from Boston to Dallas" by classifying the intent as "list flights", and filling slots "source" and "destination" with "Boston" and "Dallas", respectively.

Shallow Semantic Parsing Examples:

a) Named Entity Recognition (NER):

- **Input:** "John Smith is the CEO of Apple."
- **Output:** ["John Smith" (Person), "Apple" (Organization)]

b) Part-of-Speech (POS) Tagging:

- **Input:** "The quick brown fox jumps over the lazy dog."
- **Output:** ["The" (Article), "quick" (Adjective), "brown" (Adjective), "fox" (Noun), ...]

Semantic Parsing:

c) Dependency Parsing:

- **Input:** "The dog chased the cat."
- **Output:** ["dog" (Subject), "chased" (Verb), "cat" (Object)]

d) Semantic Role Labeling (SRL):

- **Input:** "John threw the ball."
- **Output:** ["John" (Agent), "threw" (Verb), "ball" (Theme)]

d) Information Extraction:

- **Input:** "John lives in New York and works at Google."
- **Output:** ["John" (Person), "lives" (Relation), "New York" (Location), "works" (Relation), "Google" (Organization)]

Semantic Parsing:

Here are some examples of Deep Semantic Parsing:

Example 1:
"**The company will increase its production by 20% next quarter**."

Deep Semantic Parse:

- Entities:
  - Company (Organization)
  - Production (Concept)
  - Quarter (Time)
- Predicate: Increase (Action)
- Arguments:
  - Agent: Company
  - Theme: Production
  - Quantity: 20%
  - Time: Next Quarter

- Semantic Roles:
  - Agent: Company
  - Theme: Production
  - Goal: Increase
  - Source: None
  - Time: Next Quarter

Semantic Parsing:

" **The company will increase its production by 20% next quarter**."

- Entities:
  - c (Company)
  - p (Production)
  - q (Quarter)
- Predicate: Increase
- Arguments:
  - Agent: c
  - Theme: p
  - Quantity: 20%
  - Time: q
- Semantic Roles:
  - Agent: c
  - Theme: p
  - Goal: Increase
  - Time: q

Logical Form:

$\exists e \, (\text{Increase}(e, c, p, 0.2, q) \wedge \text{Next}(q))$

Semantic Parsing:

"**John booked a flight from New York to Los Angeles**."

Deep Semantic Parse:

- Entities:
  - John (Person)
  - Flight (Event)
  - New York (Location)
  - Los Angeles (Location)
- Predicate: Book (Action)
- Arguments:
  - Agent: John
  - Theme: Flight
  - Source: New York
  - Goal: Los Angeles

- Semantic Roles:
  - Agent: John
  - Theme: Flight
  - Source: New York
  - Goal: Los Angeles

Semantic Parsing:

"**John booked a flight from New York to Los Angeles**."

- Entities:
  - j (John)
  - f (Flight)
  - ny (New York)
  - la (Los Angeles)
- Predicate: Book
- Arguments:
  - Agent: j
  - Theme: f
  - Source: ny
  - Goal: la
- Semantic Roles:
  - Agent: j
  - Theme: f
  - Source: ny
  - Goal: la

Logical Form:

$\exists e$ (Book(e, j, f, ny, la))

Semantic Parsing:

Girish Lectures
Learning from Experience, Sharing with Passion

Deep Semantic Parsing	Shallow Semantic Parsing
Goals: • Capture detailed, formal representations of meaning • Enable machines to truly understand natural language	**Goals:** • Extract specific information from text • Identify entities, relationships, and concepts
Characteristics: • **Compositional semantics:** Represents meaning as a composition of smaller parts • **Formal semantics:** Uses logical forms, lambda calculus, or other formal representations • **Inference-enabled:** Supports reasoning and inference • **Domain-independent:** Applicable across various domains	Characteristics: • **Lexical semantics:** Focuses on word-level meaning • **Shallow representation:** Limited contextual understanding • **Domain-dependent:** Often requires domain-specific training data • **Heuristic-based:** Relies on rules and heuristics

Semantic Parsing:

Girish Lectures
Learning from Experience, Sharing with Passion

Deep Semantic Parsing	Shallow Semantic Parsing
Examples: • **Logical Form:** "John loves Mary" → ∃x (love(x, John, Mary)) • **Conceptual Graphs:** Representing entities, relationships, and events • **Semantic Networks:** Capturing relationships between concepts	**Examples:** • Named Entity Recognition (NER) • Part-of-Speech (POS) Tagging • Dependency Parsing • Semantic Role Labeling (SRL)
Tools and Libraries: • CCG (Combinatory Categorial Grammar) • Lambda Calculus • Conceptual Graphs • Semantic Networks	**Tools and Libraries:** • Stanford CoreNLP • spaCy • NLTK • OpenNLP
Applications: • Natural Language Understanding • Question Answering • Textual Entailment • Dialogue Systems • Knowledge Graph Construction	**Applications:** • Information Extraction • Sentiment Analysis • Text Classification • Question Answering • Text Summarization

Semantic Parsing:

❖ Key Differences:

- **Representation:** Deep semantic parsing uses formal, compositional representations, while shallow semantic parsing uses shallow, lexical representations.
- **Contextual understanding:** Deep semantic parsing aims to capture nuanced contextual relationships, whereas shallow semantic parsing focuses on local, word-level relationships.
- **Inference:** Deep semantic parsing supports inference and reasoning, while shallow semantic parsing relies on heuristics.
- **Domain applicability:** Deep semantic parsing is more domain-independent, whereas shallow semantic parsing often requires domain-specific training data.

Semantic Parsing:

Deep semantic parsing can be challenging due to various issues:

1. Ambiguity:

- Word sense ambiguity (e.g., "bank" as financial institution or riverbank)
- Syntactic ambiguity (e.g., "The dog bit the man with the hat")
- Semantic ambiguity (e.g., "I saw the girl with the binoculars")

2. Contextual Understanding:

- Capturing context beyond sentence boundaries
- Handling coreference resolution (e.g., "he" refers to whom?)
- Identifying implicit relationships

3. Linguistic Variability:

- Handling out-of-vocabulary (OOV) words
- Dealing with linguistic diversity (e.g., dialects, slang)
- Accommodating linguistic evolution (e.g., new words, changing meanings)

Semantic Parsing:

4. Knowledge Representation:

- Representing complex semantic relationships
- Integrating external knowledge sources (e.g., ontologies)
- Handling inconsistencies and contradictions

5. Scalability and Efficiency:

- Parsing large volumes of text data
- Handling long-range dependencies
- Optimizing computational resources

6. Evaluation Metrics:

- Defining meaningful evaluation metrics
- Measuring semantic accuracy
- Comparing parser performance

Semantic Parsing:

7. Data Quality and Availability:

- Access to high-quality annotated training data
- Dealing with noisy or incomplete data
- Ensuring data diversity

8. Parser Complexity:

- Balancing parser complexity and accuracy
- Managing parser parameters and hyperparameters
- Avoiding overfitting

9. Handling Errors:

- Identifying and handling parsing errors
- Recovering from errors
- Providing meaningful error messages

Semantic Parsing:

To address these challenges, researchers and developers employ various techniques:

1. **Multi-task learning**
2. **Transfer learning**
3. **Attention mechanisms**
4. **Graph-based parsing**
5. **Neural network architectures (e.g., RNNs, CNNs)**
6. **External knowledge sources (e.g., WordNet)**
7. **Data augmentation**
8. **Active learning**
9. **Human-in-the-loop parsing**

Semantic Parsing:

Shallow semantic parsing, also known as shallow semantic analysis, faces several challenges:

1) Lexical Issues

- Word sense ambiguity (e.g., "bank" as financial institution or riverbank)
- Out-of-vocabulary (OOV) words
- Handling idioms and colloquialisms
- Dealing with typos and spelling errors

2) Syntactic Issues

- Part-of-speech (POS) tagging errors
- Sentence boundary detection
- Clause identification
- Handling long-range dependencies

Semantic Parsing:

3) Semantic Issues

- Entity disambiguation (e.g., distinguishing between multiple entities with the same name)
- Relation extraction (e.g., identifying relationships between entities)
- Handling implicit relationships
- Capturing semantic nuances (e.g., sarcasm, irony)

4) Contextual Issues

- Capturing context beyond sentence boundaries
- Handling coreference resolution (e.g., "he" refers to whom?)
- Identifying implicit context
- Dealing with ambiguity in context

Semantic Parsing:

5) Technical Issues

- Parser efficiency and scalability
- Handling large volumes of text data
- Integrating external knowledge sources
- Dealing with noisy or incomplete data

6) Evaluation Issues

- Defining meaningful evaluation metrics
- Measuring semantic accuracy
- Comparing parser performance
- Handling evaluation biases

Semantic Parsing:

To address these challenges, researchers and developers employ various techniques:

1. **Machine learning algorithms (e.g., supervised, unsupervised)**
2. **Deep learning architectures (e.g., RNNs, CNNs)**
3. **Knowledge-based approaches (e.g., ontologies)**
4. **Hybrid approaches (combining multiple techniques)**
5. **Active learning and human-in-the-loop parsing**

Semantic Interpretation:

❖Semantic parsing can be considered as part of a larger process, **semantic interpretation**, which involves various components that together let us define a representation of a text that can be fed into a computer to allow further computational manipulations and search, which are prerequisite for any language understanding system or application. The following sections talk about some of the main components of this process.

1. **Explain sentences having ambiguous meanings.** For example, it should account for the fact that the word ***bill*** in the sentence *"The bill is large"* is ambiguous in the sense that it could represent money or the beak of a bird.
2. **Resolve the ambiguities of words in context.** For example, if the same sentence is extended to form *"The bill is large but need not be paid"*, then the theory should be able to disambiguate the monetary meaning of *bill*.
3. **Identify meaningless but syntactically well-formed sentences**, such as the famous example by Chomsky: *"Colorless green ideas sleep furiously*."
4. **Identify syntactically or transformationally unrelated paraphrases of a concept having the same semantic content.** Example: *"The company will increase its production."*

Semantic Interpretation:

Some requirements for achieving a semantic representation.

- **Structural ambiguity**
- **Word Sense**
- **Entity and Event Resolution**
- **Predicate-Argument Structure**
- **Meaning Representation**

Semantic Interpretation (Structural ambiguity)

❖Structure means syntactic structure of sentences.
❖The syntactic structure means transforming the a sentence into its underlying syntactic representation and in theory of semantic interpretation refer to underlying syntactic representation.

Here are some examples of structural ambiguity:

Example 1: Prepositional Phrase Attachment
"The woman saw the man with the binoculars."

- Interpretation 1: The woman used binoculars to see the man.
- Interpretation 2: The woman saw a man who was holding binoculars.

Semantic Interpretation (Structural ambiguity)

Example 2: Modifier Attachment
"I saw the girl with the red hair."

- Interpretation 1: The girl I saw has red hair.
- Interpretation 2: I saw a girl who was with someone with red hair.

Example 3: Clause Structure
"The dog bit the man who was walking down the street."

- Interpretation 1: The dog bit a man who was walking down the street.
- Interpretation 2: The dog, which was walking down the street, bit a man.

Semantic Interpretation (Structural ambiguity)

Example 4: Coordination
"I love reading books and writing stories."

- Interpretation 1: I love reading books and I also love writing stories.
- Interpretation 2: I love reading books that are about writing stories.

Example 5: Relative Clause
"The professor who taught me is retiring."

- Interpretation 1: The professor who taught me (specific professor) is retiring.
- Interpretation 2: The professor (in general) who taught me is retiring.

Semantic Interpretation (Structural ambiguity)

To resolve structural ambiguity, linguists and NLP researchers use various techniques:

1. **Contextual information**
2. **Syntactic parsing**
3. **Semantic role labeling**
4. **Discourse analysis**
5. **World knowledge**

Some popular datasets for structural ambiguity resolution include:

1. **Penn Treebank**
2. **Stanford Parse Treebank**
3. **Google Natural Language Processing Dataset**
4. **CoNLL Shared Task**
5. **Linguistic Data Consortium**

Semantic Interpretation: (Word Sense)

- In any given language, the same word type is used in different contexts and with different morphological variants to represent different entities or concepts in the world.

- Word sense refers to the meaning or interpretation of a word in a specific context. Words can have multiple senses, leading to ambiguity.

Types of Word Senses:

1. **Polysemy:** Multiple related meanings (e.g., "bank" as financial institution or riverbank)
2. **Homonymy:** Multiple unrelated meanings (e.g., "bat" as flying mammal or sports equipment)
3. **Hyponymy:** Specific instances of a general concept (e.g., "rose" as a type of flower)
4. **Hypernymy:** General concepts encompassing specific instances (e.g., "flower" encompassing roses, daisies, etc.)
5. **Metonymy:** Related concepts (e.g., "White House" referring to the US government)

Semantic Interpretation: (Word Sense)

6. Word Sense Disambiguation (WSD)

Sentence: "The bank will finance the construction project.“ WSD: Determine whether "bank" refers to a financial institution or riverbank.

7. Contextual Word Sense : "red" (color) vs. "red" (indicating error)

Ex: "The teacher graded the papers with a red pen.“

8. Idiomatic Word Sense: "kick" (literal) vs. "kick the bucket" (idiomatic expression meaning "to die")

9. Domain-Specific Word Sense: "virus" (medical) vs. "virus" (computer science)

10. Word Sense in Sentiment Analysis: "blast" (positive) vs. "blast" (negative)

Ex: “The movie was a blast”

Semantic Interpretation: (Word Sense)

- **Word Sense Disambiguation (WSD) Techniques:**

  1. **Supervised Learning:** Training models on labeled data
  2. **Unsupervised Learning:** Clustering words based on context
  3. **Knowledge-based:** Using dictionaries, thesauri, or ontologies
  4. **Hybrid:** Combining multiple techniques

- **WSD Applications:**

  1. Text Classification
  2. Information Retrieval
  3. Question Answering
  4. Machine Translation
  5. Sentiment Analysis

Semantic Interpretation:(Word Sense)

For example, we use the word **nail** to represent a part of the human anatomy and also to represent the generally metallic object used to secure other objects.

1. He ***nailed*** the loose arm of the chair with a hammer.
2. He bought a box of ***nails*** from the hardware store.
3. He went to the beauty salon to get his ***nails*** clipped.
4. He went to get a manicure. His ***nails*** had grown very long.

Semantic Interpretation:(Entity and Event Resolution)

❖Identification of various entities that are sprinkled across the discourse using the same or different phrases.

❖Reconciling what type of entity or event is being considered, along with disambiguating various ways in which the same entity is referred to over a discourse, is critical to creating a semantic representation.

❖The predominant tasks have become popular over the years: **named entity recognition** and **coreference resolution**.

Semantic Interpretation:(Entity and Event Resolution)

Entity Resolution:

- Entity resolution involves identifying and linking mentions of entities (e.g., people, organizations, locations) across text.

Types of Entities:

1. **Named Entities (NE):** People, Organizations, Locations
2. **Nominal Entities:** Objects, Events, Concepts
3. **Pronominal Entities:** Pronouns referring to entities

Entity Resolution Techniques:

1. Rule-based approaches
2. Machine Learning (ML) models
3. Deep Learning (DL) models
4. Graph-based methods

Entity Resolution Applications:

1. Information Retrieval
2. Question Answering
3. Text Summarization
4. Sentiment Analysis
5. Knowledge Graph Construction

Semantic Interpretation:(Entity and Event Resolution)

Event Resolution:

Event resolution involves identifying and linking mentions of events (e.g., meetings, accidents, elections) across text.

Types of Events:

1. Actions (e.g., "meeting")
2. States (e.g., "being happy")
3. Events with temporal relationships (e.g., "before", "after")

Event Resolution Techniques:

1. Event extraction using patterns
2. ML models for event detection
3. DL models for event extraction
4. Graph-based methods for event coreference

Event Resolution Applications:

1. Event detection and tracking
2. Timeline construction
3. Question Answering
4. Text Summarization
5. Predictive analytics

Semantic Interpretation:(Entity and Event Resolution)

Two predominant tasks have become popular over the years:

Named entity recognition:

- Named Entity Recognition (NER) is a fundamental task in Natural Language Processing (NLP) that involves identifying and classifying entities or named entities in text.
- Named entities are real-world objects that have names, such as persons, organizations, locations, dates, numerical values, and more. NER aims to automatically locate and categorize these entities in a given text.

Example: "Apple" can be a fruit in one context and a company in another.

Coreference resolution.

- Coreference resolution is a natural language processing (NLP) task that involves determining when two or more expressions (words or phrases) in a text refer to the same entity or concept.
- This is important because in language, speakers or writers often use different words or phrases to refer to the same thing, and coreference resolution helps to establish those connections.

Semantic Interpretation:(Entity and Event Resolution)

❖Coreference resolution is particularly important for understanding the context and relationships between various elements in a text, and it plays a crucial role in tasks like text summarization, question answering, and machine translation.

Example:

- "John saw a movie. He enjoyed it."
- In this sentence, "He" refers to the same entity as "John." Coreference resolution is the process of determining that "He" and "John" both refer to the same person.

Semantic Interpretation:(Predicate-Argument Structure)

– Predicate-Argument Structure (PAS) represents the relationship between a predicate (verb or action) and its arguments (entities involved).

❖**Components:**

- ❖**Predicate:** The main action or state described in the sentence (e.g., "eat," "buy").
- ❖**Arguments:** Entities involved in the action or state (e.g., "John," "apple").
- ❖**Roles:** Semantic roles played by arguments (e.g., "agent," "theme").

❖**Predicate-Argument Structure Types:**

- ❖**Intransitive:** Single argument (e.g., "John sleeps").
- ❖**Transitive:** Two arguments (e.g., "John eats an apple").
- ❖**Ditransitive:** Three arguments (e.g., "John gives Mary an apple").

Semantic Interpretation:(Predicate-Argument Structure)

❖Semantic Roles:

- ❖**Agent:** Entity performing the action (e.g., "John" in "John eats").
- ❖**Theme:** Entity affected by the action (e.g., "apple" in "John eats").
- ❖**Goal:** Entity receiving the action (e.g., "Mary" in "John gives Mary").
- ❖**Source:** Entity originating the action (e.g., "store" in "John buys from").
- ❖**Experiencer:** Entity experiencing emotions or sensations (e.g., "John" in "John feels happy").

❖PAS Representation:

- ❖**PropBank:** Verb-centric representation (e.g., "eat-01" with ARG0=agent, ARG1=theme).
- ❖**FrameNet:** Frame-based representation (e.g., "Eating" frame with "eater," "food").
- ❖**Semantic Role Labeling (SRL):** Annotation of semantic roles.

Semantic Interpretation:(Predicate-Argument Structure)

❖Applications:

- ❖**Question Answering:** Identifying entities and relationships.
- ❖**Text Summarization:** Capturing key events and entities.
- ❖**Information Extraction:** Identifying relevant information.
- ❖**Machine Translation:** Preserving semantic relationships.
- ❖**Natural Language Generation:** Generating coherent text.

❖Challenges:

- ❖**Ambiguity:** Resolving unclear roles or relationships.
- ❖**Contextual understanding:** Capturing implicit relationships.
- ❖**Handling idioms and colloquialisms:** Accounting for non-literal language.
- ❖**Scalability:** Processing large volumes of text.

Semantic Interpretation:(Predicate-Argument Structure)

Bell Atlantic Corp. *said* it will *acquire* one of Control Data Corp.'s computer maintenance businesses.

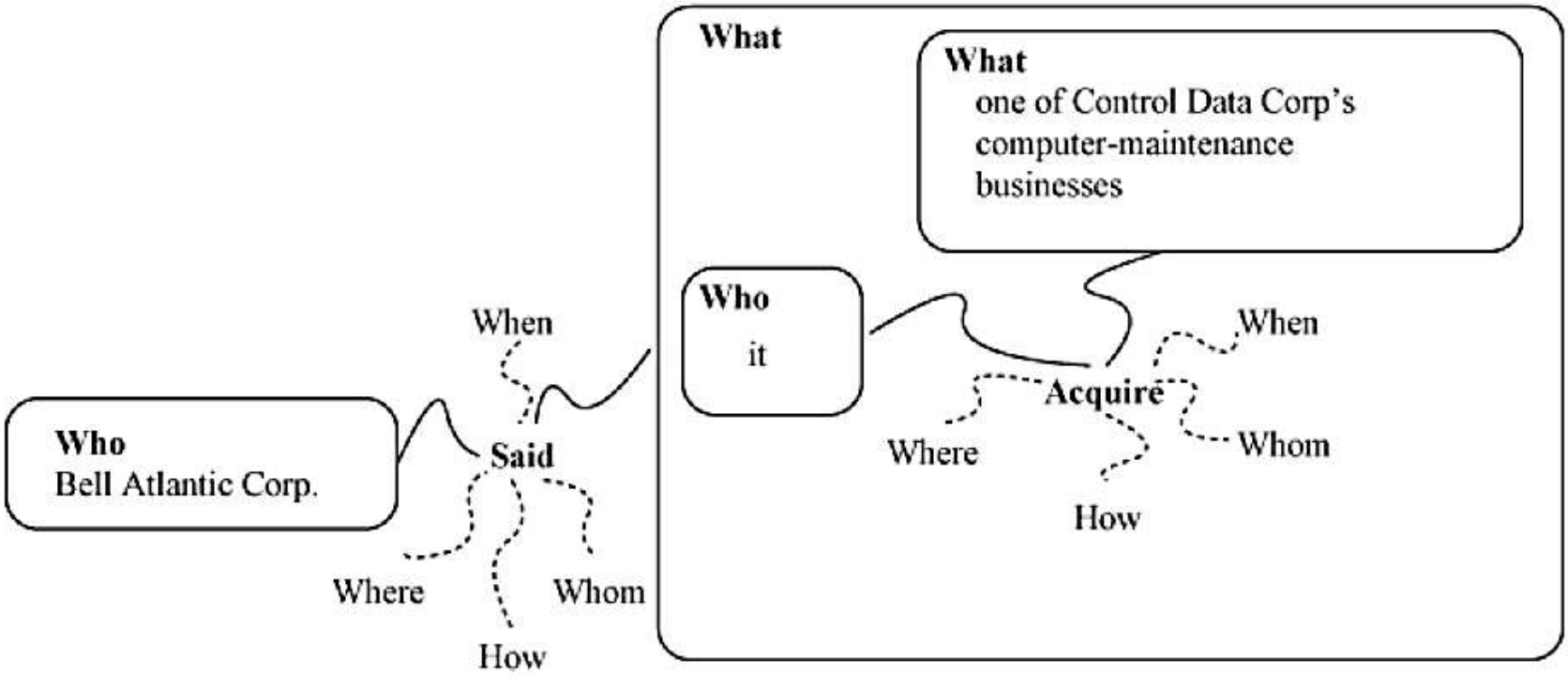


Figure 4–1. A representation of *who* did *what* to *whom*, *when*, *where*, *why*, and *how*

Semantic Interpretation(Meaning Representation)

❖The final process of the semantic interpretation is to build a semantic representation or meaning representation that can then be manipulated by algorithms to various application ends.

❖This process is sometimes called the **deep representation**.

❖Any given application, most studies in this area have been application dependent, or dependent on the domain of particular applications.

❖The following two examples show sample sentences and their meaning representations

1. If our player 2 has the ball, then position our player 5 in the midfield.

```
((bowner (player our 2)) (do (player our 5) (pos (midfield)))
```

2. Which river is the longest?

$$\text{answer}(x_1, \text{longest}(x_1\ \text{river}(x_1)))$$

Semantic Interpretation(Meaning Representation)

Meaning Representation (MR) is a formal structure for encoding semantic information in natural language.

Types of Meaning Representation:

1. **Logical Form:** Representing meaning using logical operators (e.g., predicates, quantifiers).
2. **Semantic Networks:** Representing relationships between concepts (e.g., frames, graphs).
3. **Conceptual Graphs:** Visual representation of semantic relationships.
4. **Frame Semantics:** Representing meaning using frames (e.g., situations, events).
5. **Model-Theoretic Semantics:** Interpreting meaning using mathematical models.

Components of Meaning Representation:

1. **Entities:** Objects, concepts, or individuals.
2. **Predicates:** Relations, actions, or states.
3. **Arguments:** Entities involved in predicates.
4. **Modifiers:** Attributes or properties of entities or predicates.
5. **Quantifiers:** Expressing scope or quantity (e.g., "all," "some").

Semantic Interpretation(Meaning Representation)

Formalisms for Meaning Representation:

1. **First-Order Logic (FOL):** Using predicates, variables, and quantifiers.
2. **Description Logics (DL):** Representing knowledge using concepts, roles, and individuals.
3. **Conceptual Graphs (CG):** Visual representation using graphs.
4. **Semantic Web languages (RDF, OWL):** Representing knowledge for web applications.

Applications of Meaning Representation:

1. **Natural Language Processing (NLP):** Text analysis, sentiment analysis.
2. **Artificial Intelligence (AI):** Knowledge representation, reasoning.
3. **Question Answering:** Identifying relevant information.
4. **Text Summarization:** Capturing key concepts.
5. **Machine Translation:** Preserving semantic relationships.

Semantic Interpretation(Meaning Representation)

Girish Lectures
Learning from Experience, Sharing with Passion

Example 1:
"The dog chased the cat."

Meaning Representation:

- Entities:
  - Dog (agent)
  - Cat (theme)
- Predicate: Chase (action)
- Arguments:
  - Agent: Dog
  - Theme: Cat
- Logical Form: Chase(Dog, Cat)

Example 2:
"John bought a book from Amazon."

Meaning Representation:

- Entities:
  - John (agent)
  - Book (theme)
  - Amazon (source)
- Predicate: Buy (action)
- Arguments:
  - Agent: John
  - Theme: Book
  - Source: Amazon
- Logical Form: Buy(John, Book, Amazon)

Example 3:
"The company increased its production."

Meaning Representation:

- Entities:
  - Company (agent)
  - Production (theme)
- Predicate: Increase (action)
- Arguments:
  - Agent: Company
  - Theme: Production
- Logical Form: Increase(Company, Production)

System Paradigms :

1. System Architectures

a. Knowledge based:

- As the name suggests, these systems use a predefined set of rules or a knowledge base to obtain a solution to a new problem. **(rule-based)**

b. Unsupervised:

- These systems tend to require minimal human intervention to be functional by using existing resources that can be bootstrapped for a particular application or problem domain. **(minimal human intervention)**

c. Supervised:

- Researchers create feature functions that allow each problem instance to be projected into a space of features. A model is trained to use these features to predict labels, and then it is applied to unseen data**. (manual annotation and machine learning)**

d. Semi-Supervised:

- Researchers can automatically expand the data set on which their models are trained either by employing machine-generated output directly or by bootstrapping off of an existing model by having humans correct its output. In many cases, a model from one domain is used to quickly adapt to a new domain. **(combines manual and automatic annotation)**

System Paradigms :

2. Scope

a. Domain Dependent:

- These systems are specific to certain domains, such as air travel reservations or simulated football coaching. **(specific domains, e.g., air travel)**

b. Domain Independent:

- These systems are general enough that the techniques can be applicable to multiple domains without little or no change. **(generalizable across domains)**

3. Coverage

a. Shallow:

- These systems tend to produce an intermediate representation that can then be converted to one that a machine can base its actions on. **(intermediate representation)**

b. Deep:

- These systems usually create a terminal representation that is directly consumed by a machine or application. **(terminal representation for direct machine consumption)**

Word Sense:

- Word sense refers to the particular meaning that a word has in a given context. Many words in the English language have multiple senses or meanings, and the intended sense of a word can vary depending on the context in which it is used.

- For example, the word "bank" can refer to a financial institution, the side of a river, or the act of tilting to one side.

- Understanding word sense is crucial in natural language processing and computational linguistics, where it is essential to disambiguate the meaning of words in order to accurately process and interpret language.

- Various techniques, such as context analysis and semantic disambiguation, are employed to determine the correct sense of a word in a given context.

- The study of word sense and how words acquire different meanings is known as lexical semantics. Lexical semantics explores the relationships between words and their meanings, including how word senses can change over time and how words can have different meanings in different linguistic or cultural contexts.

Word Sense:

Word sense ambiguities can be of three principal types:

(i) homonymy: Homonymy indicates that the words share the same spelling, but the meanings are quite disparate.

(ii) **polysemy:** Each homonymous partition, however, may contain finer sense nuances that could be assigned to the word depending on the context, and this phenomenon is called polysemy.

For example, these two senses of the word *bank* are orthogonal: ***financial*** bank and ***river*** bank. Further, *bank* has some other, somewhat finer, and related subsenses that indicate a collection of things: for example, financial bank and bank of clouds.

iii) categorical ambiguity : The word book can mean a **book** such as the one in which this chapter appears or to enter charges against someone in a police register. The former belongs to the grammatical category of noun, and the latter, verb.

Distinguishing between these two categories effectively helps disambiguate these two senses. Therefore, categorial ambiguity can be resolved with syntactic information (part of speech) alone, but polysemy and homonymy need more than syntax.

Word Sense:

Resources

- As with any language understanding task, the availability of resources is a key factor in the disambiguation of word senses in corpora.
- Unfortunately, the community has not seen the development of a significant amount of hand-tagged sense data—at least not until very recently.
- Early work on word sense disambiguation used machine-readable dictionaries or thesauruses as knowledge sources.
- Two prominent sources were the *Longman Dictionary of Contemporary English* (LDOCE).
- **WordNet** is a lexical database of the English language that relates words to one another in terms of synonyms, hypernyms, hyponyms, and more. It is a valuable resource in the field of natural language processing and computational linguistics.
- WordNet organizes words into sets of synonyms called **synsets.** Each synset represents a distinct concept and contains a set of words that are considered synonymous in some context. For example, the synset for "car" might include words like "automobile," "motorcar," and "machine."

Word Sense:

WordNet:

Semantic Relations: WordNet establishes various semantic relations between words. Some of the important relations include:

- **Hyponymy:** Represents a "is-a" relationship, where one word is more specific than another. For example, "rose" is a hyponym of "flower."
- **Hypernymy:** Represents a "is-a" relationship in the opposite direction. Using the previous example, "flower" is a hypernym of "rose."
- **Antonymy:** Represents words with opposite meanings, such as "hot" and "cold."

❖**Applications:** WordNet is widely used in various natural language processing applications, including information retrieval, machine translation, and sentiment analysis. It provides a structured way to understand the relationships between words and their meanings.


Thank you!!!...

Lecture-#12

Natural Language Processing

A8706

UNIT-III

Semantic Parsing

Word Sense:

Systems :

Systems into four main categories:

(i) rule based or knowledge based,
(ii) supervised,
(iii) unsupervised, and
(iv) semisupervised.

Rule based or knowledge based :

Lesk Algorithm:

❖In Natural Language Processing (NLP), word sense disambiguation (WSD) is the challenge of determining which “sense” (meaning) of a word is activated by its use in a specific context, a process that appears to be mostly unconscious in individuals.
❖Lesk Algorithm is a way of Word Sense Disambiguation.
❖The Lesk algorithm is a dictionary-based approach that is considered seminal.
❖It is founded on the idea that words used in a text are related to one another, and that this relationship can be seen in the definitions of the words and their meanings.
❖The pair of dictionary senses having the highest word overlap in their dictionary meanings are used to disambiguate two (or more) terms.
❖Michael E. Lesk introduced the Lesk algorithm in 1986 as a classic approach for word sense disambiguation in Natural Language Processing.

Lesk Algorithm:

❖The Lesk algorithm assumes that words in a given “neighborhood” (a portion of text) will have a similar theme.
❖The dictionary definition of an uncertain word is compared to the terms in its neighborhood in a simplified version of the Lesk algorithm.

Basic Lesk Algorithm implementation involves the following steps:
➢Count the number of words in the neighborhood of the word and in the dictionary definition of that sense for each sense of the word being disambiguated.
➢The sense to be picked is the one with the greatest number of items in this count.
➢The core of the algorithm is that the sense of a word in a given context is most likely to be the dictionary sense whose terms most closely overlap with the terms in the context.

Lesk Algorithm:

Algorithm 4–1. Pseudocode of the simplified Lesk algorithm The function COMPUTEOVERLAP returns the number of words common to the two sets

---

Procedure: SIMPLIFIED_LESK(word, sentence) **returns** best sense of *word*

```
 1: best-sense ← most frequent sense of word
 2: max-overlap ← 0
 3: context ← set of words in sentence
 4: for all sense ∈ senses of word do
 5:     signature ← set of words in gloss and examples of sense
 6:     overlap ← COMPUTEOVERLAP(signature, context)
 7:     if overlap gt max-overlap then
 8:         max-overlap ← overlap
 9:         best-sense ← sense
10:     end if
11: end for
12: return best-sense
```

Lesk Algorithm:


```
import nltk
from nltk.corpus import wordnet

def compute_overlap(signature, context):
    return len(set(signature) & set(context))



def simplified_lesk(word, sentence):
    # Get wordnet senses for the word
    senses = wordnet.synsets(word)

    # Initialize best-sense and max-overlap
    best_sense = senses[0]  # most frequent sense
    max_overlap = 0

    # Extract context words
    context = set(sentence.split())
```

```
 # Iterate through senses
    for sense in senses:
        # Extract signature words
        signature = set(sense.definition().split()) |
set(sense.examples())

        # Compute overlap
        overlap = compute_overlap(signature, context)

# Update max-overlap and best-sense
        if overlap > max_overlap:
            max_overlap = overlap
            best_sense = sense

    return best_sense

# Example usage
word = "bank"
sentence = "The bank is located near the river."
print(simplified_lesk(word, sentence))
```

Output:-

Synset('bank.n.07')

Lesk Algorithm:

Breakdown the output :-

Synset: A WordNet synset, representing a set of synonyms.

- bank: The word being disambiguated.
- .n.: Indicates the part of speech (noun).
- 07: The sense number (7th noun sense).

WordNet Sense:

According to WordNet, **bank.n.07** corresponds to:-

Definition: "a financial institution that provides banking services“

- **Synonyms:** {bank, banking concern, banking institution}
- Example Sentences:-
- "The bank is located downtown.“
- "I went to the bank to deposit my paycheck."

WordNet lists multiple senses for "bank", including:

1. **bank.n.01:** riverbank
2. **bank.n.02:** financial institution (more general)
3. **bank.n.03:** storage location (e.g., blood bank)
4. **bank.n.04:** sloping land (e.g., riverbank)
5. **bank.n.05:** turn or incline (e.g., bank a plane)
6. **bank.n.06:** supply or stockpile (e.g., bank votes)
7. **bank.n.07:** financial institution (specifically providing banking services)
8. **bank.n.08:** row or series (e.g., bank of switches)

The 7th noun sense (**bank.n.07**) is a specific subclass of financial institutions, emphasizing banking services.

Lesk Algorithm:

Why did Lesk return bank.n.07?
Possible reasons:

1. **Insufficient contextual information:** Lesk might not have considered the phrase **"near the river"** strongly enough to override the more frequent financial institution sense.
2. **Weighted sense probabilities:** Lesk's sense ranking might be biased towards more common senses (financial institution).
3. **Limited knowledge:** Lesk's training data or WordNet might not accurately capture nuances of geographic contexts.

Lesk Algorithm:

Why did Lesk return bank.n.07?

Possible reasons:

1. **Insufficient contextual information:** Lesk might not have considered the phrase **"near the river"** strongly enough to override the more frequent financial institution sense.
2. **Weighted sense probabilities:** Lesk's sense ranking might be biased towards more common senses (financial institution).
3. **Limited knowledge:** Lesk's training data or WordNet might not accurately capture nuances of geographic contexts.

Lesk Algorithm:

Girish Lectures
Learning from Experience, Sharing with Passion

For Example:

word = "spring"
sentence = "The spring in the garden is beautiful."
print(simplified_lesk(word, sentence))

Output:

Synset('spring.n.01')

Here are the possible senses for the word "spring" from WordNet:

Noun Senses (11)

1. **spring.n.01:** A season of the year (typically from March to May in the Northern Hemisphere)
2. **spring.n.02:** A coiled metal object that stores energy (e.g., spring in a watch)
3. **spring.n.03:** A source of water (e.g., natural spring)
4. **spring.n.04:** A type of motion (e.g., springing into action)
5. **spring.n.05:** A device that releases or absorbs energy (e.g., spring in a vehicle suspension)
6. **spring.n.06:** A type of mattress or cushion (e.g., spring mattress)
7. **spring.n.07:** A type of geological formation (e.g., spring-fed river)
8. **spring.n.08:** A type of mechanical device (e.g., spring-loaded mechanism)
9. **spring.n.09:** A type of electrical device (e.g., spring-contact switch)
10. **spring.n.10:** A type of athletic event (e.g., springboard diving)
11. **spring.n.11:** A type of spiritual renewal (e.g., spiritual spring)

Lesk Algorithm:

Verb Senses (7)

1. **spring.v.01:** To move suddenly and quickly (e.g., spring into action)
2. **spring.v.02:** To release or absorb energy (e.g., spring a trap)
3. **spring.v.03:** To originate or arise (e.g., spring from a idea)
4. **spring.v.04:** To become active or vigorous (e.g., spring to life)
5. **spring.v.05:** To produce or create (e.g., spring forth new ideas)
6. **spring.v.06:** To recover or rebound (e.g., spring back from adversity)
7. **spring.v.07:** To leak or flow (e.g., spring a leak)

Adjective Senses (4)

1. **spring.a.01:** Relating to the season of spring
2. **spring.a.02:** Characterized by sudden or rapid movement
3. **spring.a.03:** Relating to a coiled metal object
4. **spring.a.04:** Relating to spiritual renewal

Adverb Senses (1)

1. **spring.adv.01**: In a sudden or rapid manner

Supervised :

Features We discuss a more commonly found subset of features that have been useful in supervised learning of word sense. These are not exhaustive by any means, but ones that have been time-tested, and provide a very good base that can be used to achieve nearly state-of-the-art performance.

Lexical context—This feature comprises the words and lemmas of words occurring in the entire paragraph or a smaller window of usually five words.

Classifier Probably the most common and high-performing classifiers are support vector machines (SVMs) and maximum entropy (MaxEnt) classifiers. Many good-quality, freely available distributions of each are available and can be used to train word sense disambiguation models. Typically, because each lemma has a separate sense inventory, it is almost always the case that a separate model is trained for each lemma and POS combination (i.e., if the language, as in the case of English, has separate sense inventories for various parts of speech).

Supervised :

Parts of speech—This feature comprises the POS information for words in the window surrounding the word that is being sense tagged.

Bag of words context—This feature comprises using an unordered set of words in the context window. A threshold is typically tuned to include the most informative words in the larger context.

Local collocations—Local collocations are an ordered sequence of phrases near the target word that provide semantic context for disambiguation. Usually, a very small window of about three tokens on each side of the target word, most often in contiguous pairs or triplets, are added as a list of features. For example, if the target word is *w*, then $C_{i,\ j}$ would be a collocation where *i* and *j* refer to the start and offsets with re- spect to the word *w*. A positive sign indicates words on the right, and a negative sign indicates words on the left of the target.

Supervised :

Syntactic relations—If the parse of the sentence containing the target word is available, then we can use syntactic features.

Topic features—The broad topic, or domain, of the article that the word belongs to is also a good indicator of what sense of the word might be most frequent.

Voice of the sentence—This ternary feature indicates whether the sentence in which the word occurs is a passive, semipassive, or active sentence.

Presence of subject/object—This binary feature indicates whether the target word has a subject or object. Given a large amount of training data, we could also use the actual lexeme and possibly the semantic roles rather than the syntactic subject/ objects.

Prepositional phrase adjunct—This feature indicates whether the target word has a prepositional phrase, and if so, selects the head of the noun phrase inside the prepositional phrase.

Supervised :

Sentential complement—This binary feature indicates whether the word has a sentential complement.

Algorithm 4–2. Rules for selecting syntactic relations as features

1: **if** *w* is a *noun* **then**
2: *select* parent head word (*h*)
3: *select* part of speech of *h*
4: *select* voice of *h*
5: *select* position of *h* (left, right)
6: **else if** *w* is a *verb* **then**
7: *select* nearest word *l* to the left of *w* such that *w* is the parent head word of *l*
8: *select* nearest word *r* to the right of *w* such that *w* is the parent head word of *r*
9: *select* part of speech of *l*
10: *select* part of speech of *r*
11: *select* part of speech of *w*
12: *select* voice of *w*
13: **else if** *w* is a *adjective* **then**
14: *select* parent head word (*h*)
15: *select* part of speech of *h*
16: **end if**

Supervised :

From the output, we can conclude:

Syntactic Analysis:

1. **Word Position:** "**bloomed**" is in the middle of the sentence ("early" position).
2. **Left Context:** "**The beautiful flower**" provides context for the subject.
3. **Right Context:** "**in the garden**" provides context for the location.

Semantic Insights:

1. **Action:** "**bloomed**" indicates a change of state (flower blooming).
2. **Subject:** "**flower**" is the main entity performing the action.
3. **Location:** "**garden**" provides the setting for the action.

Pragmatic Implications:

1. **Topic:** The sentence focuses on the flower's blooming.
2. **Tone:** Neutral, descriptive tone.
3. **Purpose:** Informative, possibly descriptive or narrative.

Supervised :

With the parent head, we can infer:

Syntactic Relationships:

1. **Dependency:** The parent head ("**flower**") is the word that "**bloomed**" depends on.
2. **Modification:** "**Bloomed**" modifies or describes the state of "**flower**".
3. **Argument Structure:** "**Flower**" is the main argument of the verb "**bloomed**".

Semantic Insights:

1. **Entity:** "**Flower**" is the main entity involved in the action.
2. **Action Target:** The action of **blooming** is targeted at the flower.
3. **Semantic Role:** "**Flower**" plays the role of the patient or theme.

Discourse Structure:

1. **Topicality:** "**Flower**" is likely the topic of the sentence.
2. **Focus:** The sentence focuses on the flower's blooming.

Pragmatic Implications:

1. **Emphasis:** The sentence emphasizes the flower's state change.
2. **Contextualization:** The sentence provides context for the flower's situation.

Supervised :

Named entity—This feature is the named entity of the proper nouns and certain types of common nouns.

WordNet—WordNet synsets of the hypernyms of head nouns of the noun phrase arguments of verbs and prepositions.

Path—This feature is the path from the target verb to the verb's arguments.

Subcategorization—The subcategorization frame is essentially the string formed by joining the verb phrase type with that of its children.

Unsupervised :

Advances in word-sense disambiguation are being held back because there isn't enough labeled training data to build classifiers for every possible meaning of each word in a language. There are a few solutions to this problem:

- Devise a way to cluster instances of a word so that each cluster effectively constrains the examples of the word to a certain sense. This could be considered **sense induction** through clustering.

- Use some metrics to identify the proximity of a given instance with some sets of known senses of a word and select the closest to be the sense of that instance.

- Start with **seeds** of examples of certain senses, then iteratively grow them to form clusters.

Unsupervised :

Three key solutions to address the challenge of limited labelled training data in Word Sense Disambiguation (WSD):

1. Sense Induction through Clustering

Cluster instances of a word to group similar contexts/senses together. This approach:

- Reduces need for labeled data
- Identifies patterns in unlabeled data
- Can be used as preprocessing for supervised WSD

Techniques:

- K-Means
- Hierarchical Clustering
- DBSCAN
- Non-parametric Bayesian methods (e.g., Dirichlet Process Gaussian Mixture Models)

Unsupervised :

2. Metric-based Sense Identification

Measure proximity between instances and known sense clusters. This approach:

- Utilizes labeled data for known senses
- Identifies closest sense for new instances
- Can incorporate multiple metrics (e.g., semantic similarity, context overlap)

Metrics:

- Cosine similarity
- Jaccard similarity
- Word embeddings (e.g., Word2Vec, GloVe)
- Semantic similarity measures (e.g., WordNet-based)

Unsupervised :

3. Seed-Based Sense Expansion

Start with labeled examples (seeds) and iteratively grow clusters. This approach:

- Leverages labeled data for initial seeds
- Expands clusters through similarity measures
- Refines sense boundaries through iteration

Techniques:

- K-Nearest Neighbors (KNN)
- Support Vector Machines (SVM)
- Graph-based methods (e.g., Graph-Based Semi-Supervised Learning)
- Active learning for selective sampling

Unsupervised :

Additional strategies to address data scarcity:

- **Transfer learning:** Utilize pre-trained models and fine-tune for WSD
- **Multi-task learning:** Jointly learn WSD with related tasks (e.g., part-of-speech tagging)
- **Data augmentation:** Generate artificial examples through perturbation or paraphrasing
- **Weak supervision:** Use weakly labelled data or distant supervision

To further explore these solutions, we can consider by implementing the following:

1. Implementing clustering algorithms for sense induction
2. Designing metric-based sense identification systems
3. Developing seed-based sense expansion methods
4. Integrating transfer learning and multi-task learning

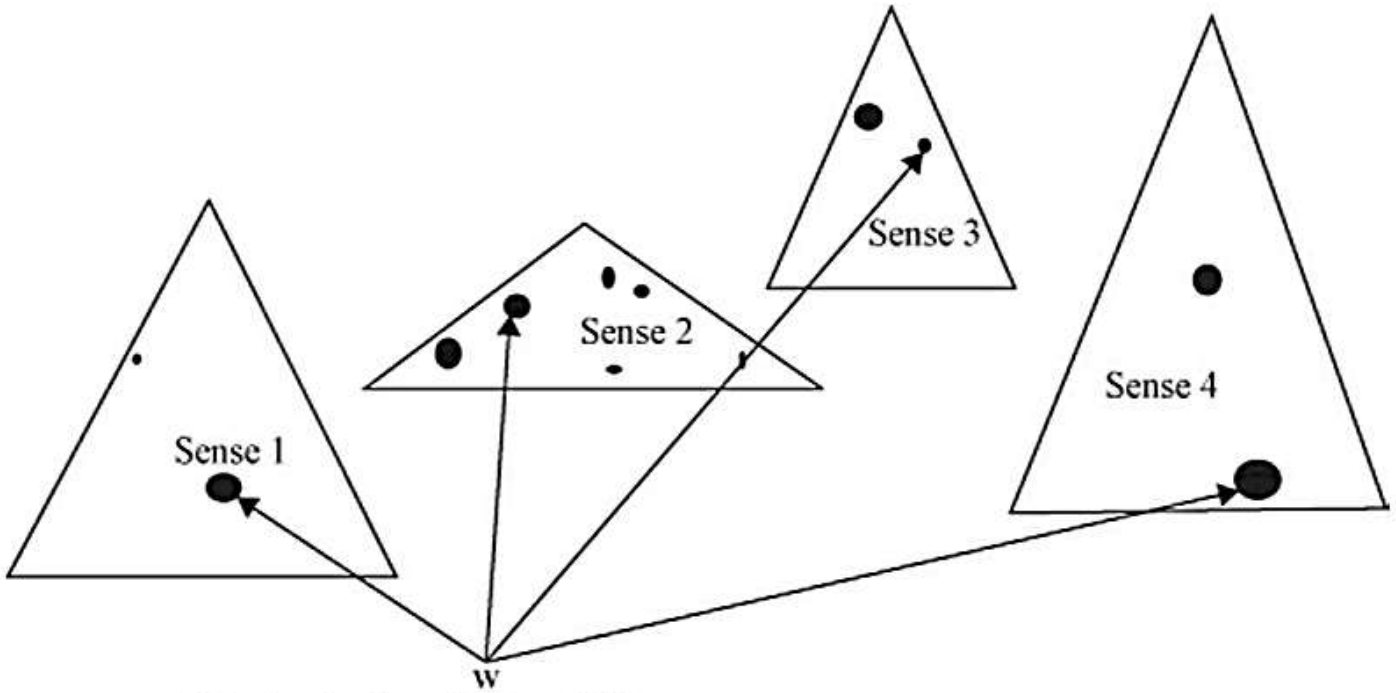
Unsupervised :

- Some of the Category of algorithms that use some form of distance measure to identify senses and introduced a metric for computing the shortest distance between the two pairs of senses in WordNet.

- This metric assumes that multiple co-occurring words are likely to exhibit senses that would minimize the distance in a semantic network of hierarchical relations.

- They proposed a new measure of semantic similarity: **information content** in an **IS-A** taxonomy which produces much better results than the edge-counting measure. Further refined this measure, calling it **conceptual density**.

Unsupervised :



Word to be disambiguated: W
Context words: w1 w2 w3 w4

$$CD(c,m) = \frac{\sum_{i=0}^{m-1} \text{hyponyms}^{i-0.20}}{\text{descendants}_c}$$

Unsupervised :

$$CD(c, m) = \frac{\sum_{i=0}^{m-1} \text{hyponyms}^{i^{-0.20}}}{\text{descendants}_c}$$

The formula represents the ***Conceptual Density (CD)*** of a concept *c* given a certain number of levels mmm in a taxonomy or hierarchy. Here's a breakdown of each component:

1.CD(c, m): This is the Conceptual Density of the concept *c* for *m* levels in the hierarchy. Conceptual Density essentially quantifies how "**dense**" or "**specific**" a concept is based on its hierarchical structure.

2.Hyponyms: In this formula, the *hyponyms* are specific concepts that fall under the broader concept c in a taxonomy. For instance, if *c* is "**animal**," a hyponym could be "**dog**," as it's a more specific type of animal.

3.∑: We sum up the values of the hyponyms from level 0 up to *m−1*, where each level *i* is weighted by the exponent −0.20. The exponent reduces the impact of hyponyms from higher levels, giving more importance to those closer to the root concept *c*.

4. Descendants: This term represents the total number of descendant concepts under *c* in the hierarchy. It serves as the denominator to normalize the density value.

Unsupervised :

$$CD(c,m) = \frac{\sum_{i=0}^{m-1} \text{hyponyms}^{i^{-0.20}}}{\text{descendants}_c}$$

- **descendants** refer to all the concepts that fall under a given concept *c* in a hierarchical structure or taxonomy. This could include any nodes that are lower or more specific than *c* within the structure.

- For example, if *c* is a general category like "animal," then its descendants could be all specific types of animals, including mammals, birds, reptiles, and all further subcategories (e.g., "dog" under "mammal," or "sparrow" under "bird").

Descendants can be :

- **Direct hyponyms**: These are the immediate subcategories of *c*. If *c* is "animal," a direct hyponym might be "mammal."
- **Indirect hyponyms**: These are subcategories that fall further down in the hierarchy, but are still conceptually under *c*. For example, "dog" would be an indirect hyponym of "animal" via "mammal."
- **All levels of subcategories**: Since this formula considers up to m levels of hyponyms, descendants here would include all concepts within those mmm levels under *c*.

Semisupervised :

- The next category of algorithms we look at are those that start from a small seed of examples and an iterative algorithm that identifies more training examples using a classifier.

- This additional, automatically labeled data can then be used to augment the training data of the classifier to provide better predictions for the next selection cycle, and so on.

- The **Yarowsky algorithm** is the classic case of such an algorithm and was seminal in introducing semisupervised methods to the word sense disambiguation problem.

- The algorithm is based on the assumption that two strong properties are exhibited by corpora:

One sense per collocation: Syntactic relationship and the types of words occurring nearby a given word tend to provide a strong indication as to the sense of that word.

One sense per discourse: Usually, in a given discourse, all instances of the same lemma tend to invoke the same sense.

Semisupervised :

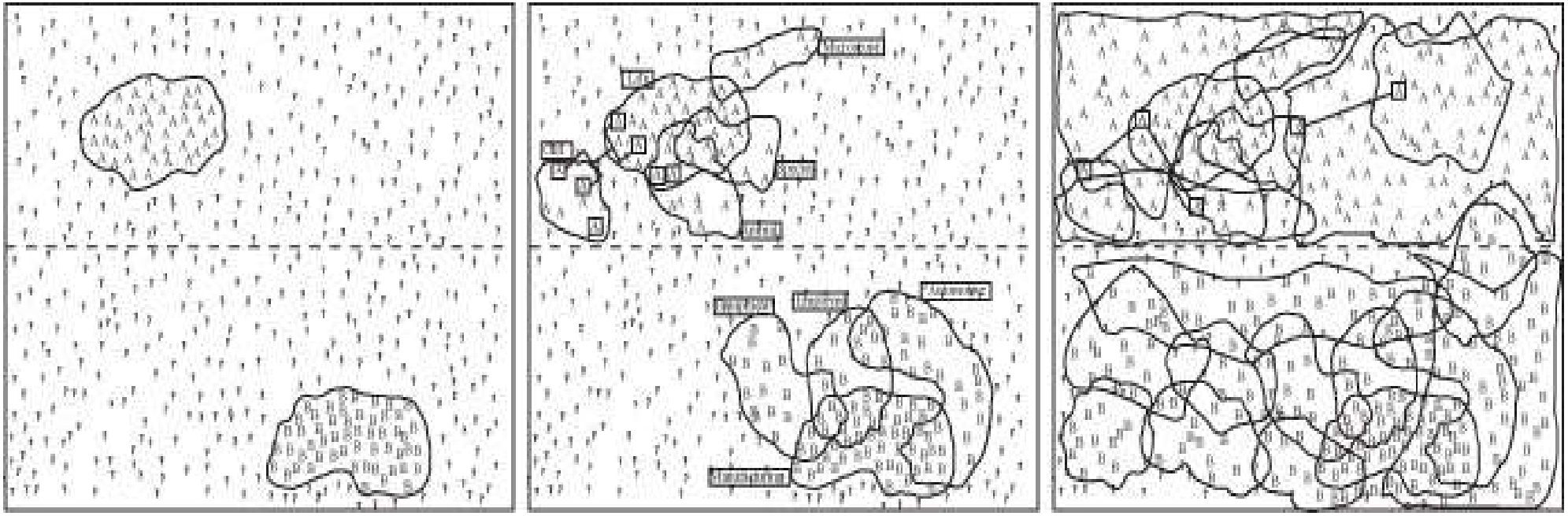


Figure 4–6. The three stages of the Yarowsky algorithm

Semisupervised :

Figure 4–6 shows the three stages of the algorithm.
In the first box, *life* and *manufacturing* are used as collocates to identify the two senses of *plant*.
Then, in the next iteration, a new collocate *cell* is identified, and the final block shows the small residual remaining at the end of the algorithm cycle.

Semisupervised :

Step 1. In a sufficiently large corpus, identify all the instances of a particular polysemous word that needs to be disambiguated, storing its context alongside.
Step 2. Identify a small set of instances that are strongly representative of one of the senses of the word. This can either be done in a completely unsupervised fashion by identifying collocations that give a strong indication of the sense usage for the word under consideration or by manually tagging a small portion of the data. In this example, we assume a polysemous word with only two senses, but this algorithm can be extended to n senses.
Step 3.

Step 3a. Train a supervised classifier on this set of examples.

Step 3b. Using these classifiers, classify the remaining instances of the word in the corpus and select those that are classified above a certain level of confidence.

Step 3c. Filter out the possible misclassifications using one sense per discourse constraint, and identify possible new collocations to be added to the list of seed collocations.

Step 3d. Repeat step 3 iteratively, thereby slowly shrinking the residual.

Step 4. Stop. At some point, a small, stable residual will remain.
Step 5. The trained classifier can now be used to classify new data, and that in turn can be used to annotate the original corpus with sense tags and probabilities.

Semisupervised : Yarowsky Algorithm

- **S1**: Botanical sense (e.g., "The plant grew in the garden").
- **S2**: Industrial sense (e.g., "The plant manufactures cars").

- A small **labeled dataset** with manually tagged sentences.
- A larger **unlabeled dataset** to classify.
- Contextual features (words within a ±3-word window around "plant") to guide classification.

The Yarowsky algorithm will:

1. Start with a seed set of labeled data.
2. Extract features and build a classifier (using log-likelihood ratios).
3. Iteratively apply the classifier to unlabeled data, adding high-confidence labels to the training set.
4. Update the classifier and repeat until convergence.

Semisupervised : Yarowsky Algorithm

Step 1: Initial Labeled Data

Labeled dataset (4 sentences):

L1: "The **plant** grew in the garden." → **S1 (Botanical)**

L2: "Water the **plant** daily." → **S1 (Botanical)**

L3: "The **plant** manufactures cars." → **S2 (Industrial)**

L4: "The **plant** is near the city." → **S2 (Industrial)**

Unlabeled dataset (4 sentences):

U1: "The **plant** needs sunlight."

U2: "The **plant** employs 500 workers."

U3: "This **plant** blooms in spring."

U4: "The **plant** operates 24/7."

Semisupervised : Yarowsky Algorithm

Step 2: Extract Contextual Features

We extract features (words within ± 3 words of "plant") from the labeled data. For simplicity, we'll focus on single words as features.

L1: "The **plant** grew in the garden."

Features: {grew, garden}

L2: "Water the **plant** daily."

Features: {water, daily}

L3: "The **plant** manufactures cars."

Features: {manufactures, cars}

L4: "The **plant** is near the city."

Features: {near, city}

Semisupervised : Yarowsky Algorithm

Initial feature sets:
S1 (Botanical): {grew, garden, water, daily}
S2 (Industrial): {manufactures, cars, near, city}

Step 3: Calculate Feature Weights (Log-Likelihood Ratios)

The Yarowsky algorithm often uses a decision list based on log-likelihood ratios to rank features by their discriminative power. For each feature, we compute the probability of it appearing with each sense and calculate the log-likelihood ratio to determine which sense it favors.

Formula for Log-Likelihood Ratio: For a feature f, the log-likelihood ratio for favoring S1 over S2 is:

$$\text{LLR}(f) = \log\left(\frac{P(f|S1)}{P(f|S2)}\right)$$

Where:

- P(f|S1): Probability of feature f appearing in a sentence with sense S1.
- P(f|S2): Probability of feature f appearing in a sentence with sense S2.

To avoid zero probabilities, we apply **add-one (Laplace) smoothing**. If a feature doesn't appear with a sense, we assign it a small count (1) to avoid division by zero.

Semisupervised : Yarowsky Algorithm

Counts:

Total labeled sentences: 4 (2 for S1, 2 for S2).

Feature counts:

grew: Appears in 1 S1 sentence, 0 S2 sentences.

garden: Appears in 1 S1 sentence, 0 S2 sentences.

water: Appears in 1 S1 sentence, 0 S2 sentences.

daily: Appears in 1 S1 sentence, 0 S2 sentences.

manufactures: Appears in 0 S1 sentences, 1 S2 sentence.

cars: Appears in 0 S1 sentences, 1 S2 sentence.

near: Appears in 0 S1 sentences, 1 S2 sentence.

city: Appears in 0 S1 sentences, 1 S2 sentence.

Semisupervised : Yarowsky Algorithm

Smoothing:

Vocabulary size (unique features): 8 (grew, garden, water, daily, manufactures, cars, near, city).

For each sense, add 1 to the count of each feature and add 8 (vocabulary size) to the denominator for smoothing.

Probabilities with Smoothing:

Total feature occurrences in S1: 4 (grew, garden, water, daily).

Total feature occurrences in S2: 4 (manufactures, cars, near, city).

Smoothed counts:

For S1: Total count = 4 (actual) + 8 (smoothing for 8 features) = 12.

For S2: Total count = 4 (actual) + 8 (smoothing) = 12.

Semisupervised : Yarowsky Algorithm

For each feature:

- $P(f|S1) = \frac{\text{Count}(f,S1)+1}{\text{Total count for S1}+\text{Vocabulary size}} = \frac{\text{Count}(f,S1)+1}{12}$
- $P(f|S2) = \frac{\text{Count}(f,S2)+1}{\text{Total count for S2}+\text{Vocabulary size}} = \frac{\text{Count}(f,S2)+1}{12}$

LLR Calculations:

- **grew**:
  - $P(\text{grew}|S1) = \frac{1+1}{12} = \frac{2}{12} = 0.1667$
  - $P(\text{grew}|S2) = \frac{0+1}{12} = \frac{1}{12} = 0.0833$
  - $\text{LLR}(\text{grew}) = \log_{10}\left(\frac{0.1667}{0.0833}\right) = \log_{10}(2) \approx 0.3010$ (favors S1)
- **garden**:
  - $P(\text{garden}|S1) = \frac{1+1}{12} = 0.1667$
  - $P(\text{garden}|S2) = \frac{0+1}{12} = 0.0833$
  - $\text{LLR}(\text{garden}) = \log_{10}(2) \approx 0.3010$ (favors S1)

Semisupervised : Yarowsky Algorithm

- **water**:
  - $P(\text{water}|S1) = \frac{1+1}{12} = 0.1667$
  - $P(\text{water}|S2) = \frac{0+1}{12} = 0.0833$
  - $\text{LLR}(\text{water}) = \log_{10}(2) \approx 0.3010$ (favors S1)
- **daily**:
  - $P(\text{daily}|S1) = \frac{1+1}{12} = 0.1667$
  - $P(\text{daily}|S2) = \frac{0+1}{12} = 0.0833$
  - $\text{LLR}(\text{daily}) = \log_{10}(2) \approx 0.3010$ (favors S1)
- **manufactures**:
  - $P(\text{manufactures}|S1) = \frac{0+1}{12} = 0.0833$
  - $P(\text{manufactures}|S2) = \frac{1+1}{12} = 0.1667$
  - $\text{LLR}(\text{manufactures}) = \log_{10}\left(\frac{0.0833}{0.1667}\right) = \log_{10}(0.5) \approx -0.3010$ (favors S2)

Semisupervised : Yarowsky Algorithm

- **cars**:
  - $P(\text{cars}|S1) = \frac{0+1}{12} = 0.0833$
  - $P(\text{cars}|S2) = \frac{1+1}{12} = 0.1667$
  - $\text{LLR}(\text{cars}) = \log_{10}(0.5) \approx -0.3010$ (favors S2)
- **near**:
  - $P(\text{near}|S1) = \frac{0+1}{12} = 0.0833$
  - $P(\text{near}|S2) = \frac{1+1}{12} = 0.1667$
  - $\text{LLR}(\text{near}) = \log_{10}(0.5) \approx -0.3010$ (favors S2)
- **city**:
  - $P(\text{city}|S1) = \frac{0+1}{12} = 0.0833$
  - $P(\text{city}|S2) = \frac{1+1}{12} = 0.1667$
  - $\text{LLR}(\text{city}) = \log_{10}(0.5) \approx -0.3010$ (favors S2)

Semisupervised : Yarowsky Algorithm

Feature	LLR	Favors
grew	0.3010	S1
garden	0.3010	S1
water	0.3010	S1
daily	0.3010	S1
manufactures	-0.3010	S2
cars	-0.3010	S2
near	-0.3010	S2
city	-0.3010	S2

Semisupervised : Yarowsky Algorithm

Classify Unlabeled Sentences:

U1: "The **plant** needs sunlight."

- Features: {needs, sunlight}
- Decision list check: Neither "needs" nor "sunlight" is in the list.
- **Action**: Since no features match, we cannot confidently classify U1 with the current decision list. Mark as low confidence or skip for now.

U2: "The **plant** employs 500 workers."

- Features: {employs, workers}
- Decision list check: Neither "employs" nor "workers" is in the list.
- **Action**: No match, skip or mark as low confidence.

U3: "This **plant** blooms in spring."

- Features: {blooms, spring}
- Decision list check: Neither "blooms" nor "spring" is in the list.
- **Action**: No match, skip or mark as low confidence.

U4: "The **plant** operates 24/7."

- Features: {operates}
- Decision list check: "operates" is not in the list.
- **Action**: No match, skip or mark as low confidence.

Semisupervised : Yarowsky Algorithm

Hypothesize Senses for Unlabeled Data (based on semantic clues):

U1: "The **plant** needs sunlight."
"Sunlight" is strongly associated with botanical contexts (e.g., plants needing light to grow).
Hypothesized sense: **S1 (Botanical)**.
New feature: {sunlight}

U2: "The **plant** employs 500 workers."
"Employs" and "workers" suggest a workplace or factory.
Hypothesized sense: **S2 (Industrial)**.
New features: {employs, workers}

U3: "This **plant** blooms in spring."
"Blooms" and "spring" are strongly botanical (flowering plants).
Hypothesized sense: **S1 (Botanical)**.
New features: {blooms, spring}

U4: "The **plant** operates 24/7."
"Operates" suggests machinery or a facility, leaning industrial.
Hypothesized sense: **S2 (Industrial)**.
New feature: {operates}

Semisupervised : Yarowsky Algorithm

Updated Feature Sets (tentative, pending confidence check):

S1 (Botanical): {grew, garden, water, daily, sunlight, blooms, spring}
S2 (Industrial): {manufactures, cars, near, city, employs, workers, operates}

Recalculate LLRs for Expanded Feature Set

Incorporate the new features and recalculate LLRs to update the decision list. Assume each new feature appears once in the hypothesized sense (based on the unlabelled data).

Updated Counts:

Vocabulary size: 8 (original: grew, garden, water, daily, manufactures, cars, near, city) + 6 (new: sunlight, blooms, spring, employs, workers, operates) = 14.

S1 features: 4 (original: grew, garden, water, daily) + 3 (new: sunlight, blooms, spring) = 7.
S2 features: 4 (original: manufactures, cars, near, city) + 3 (new: employs, workers, operates) = 7.

Smoothed totals:

- S1: 7 (actual) + 14 (smoothing for vocabulary) = 21.
- S2: 7 (actual) + 14 (smoothing) = 21.

Semisupervised :

Step 1: Extract Instances
In a large corpus, find all instances of the word "bass" along with its surrounding context. For example:

1."The bass swam quickly in the lake."
2."He played a beautiful melody on the bass."
3."She caught a large bass during the fishing trip."
4."The bass guitar sounded deep and resonant."

Step 2: Create Seed Instances

Choose a few instances that are clearly representative of each sense, either by manual annotation or through identifying typical collocations. Here, we could pick:

Fish sense: "swam in the lake," "caught," "fishing trip"
Instrument sense: "melody," "guitar," "played"

These collocations will help to distinguish between the meanings of "bass."

Step 3: Train and Iteratively Refine the Classifier
Step 3a: Train a supervised classifier using the seed examples:

Fish context: "bass" + (collocations like "lake," "fishing," "swam," "caught")
Instrument context: "bass" + (collocations like "played," "guitar," "melody")

Semisupervised :

- **Step 3b**: Apply the classifier to the remaining examples in the corpus, identifying additional instances with high confidence. For example, if the context includes "pond" or "river," it's likely to be the fish sense; if it includes "song" or "strings," it's likely to be the instrument sense.
- **Step 3c**: Filter misclassifications. For instance, if the classifier misclassified an instance with "fishing gear" as the instrument sense, that could be manually corrected or adjusted based on a rule such as "one sense per discourse."
- **Step 3d**: Repeat Step 3 until the classifier can consistently and confidently classify "bass" with minimal errors.

Step 4: Stop When Stable

When only a few uncertain cases remain, the classifier is likely stable enough to handle most occurrences of "bass" in the corpus.

Step 5: Annotate the Corpus

Use the trained classifier to annotate all instances of "bass" in the corpus with either the fish or instrument sense, along with a confidence probability.

Software :

Several software programs are made available by the research community for word sense disambiguation, ranging from similarity measure modules to full disambiguation systems. It is not possible to list all of them here, so we list a selected few.

IMS (It Makes Sense) **http://nlp.comp.nus.edu.sg/software** This is a complete word sense disambiguation system.

WordNet-Similarity-2.05 http://search.cpan.org/dist/WordNet- Similarity These WordNet Similarity modules for Perl provide a quick way of computing various word similarity measures.

WikiRelate! http://www.h-its.org/english/research/nlp/download/wiki_pediasimilarity.php This is a word similarity measure based on the categories in Wikipedia.


Thank you!!!...

Lecture-#13

Natural Language Processing

A8706

UNIT-III

Semantic Parsing

Predicate-Argument Structure:

❖Shallow semantic parsing, or what is popularly known today as **semantic role labeling**, is the process of identifying the various arguments of predicates in a sentence.

❖The linguistic community has been debating for a few decades over what constitutes the set of arguments and what the granularity of such argument labels should be for various predicates, which in turn can be the verbs, nouns, adjectives, and prepositions in a sentence.

❖The late 1990s saw the emergence of two important corpora that are semantically tagged. One is **FrameNet** and the other is **PropBank.**

❖FrameNet is based on the theory of **frame semantics**, where a given predicate invokes a **semantic frame**, thus instantiating some or all of the possible semantic roles belonging to that frame.

❖**PropBank**, short for Proposition Bank, is a resource in natural language processing (NLP) that provides information about the semantic roles of verbs in English sentences.

❖It annotates texts with information about basic semantic propositions, including the arguments (participants) that fill the various roles associated with a verb.

FrameNet:

❖**FrameNet** contains frame-specific semantic annotation of a number of predicates in English.

❖ It contains tagged sentences extracted from the British National Corpus (BNC). The process of **FrameNet** annotation consists of identifying specific semantic frames and creating a set of frame- specific roles called **frame elements**.

❖Then, a set of predicates that instantiate the semantic frame, irrespective of their grammatical category, are identified, and a variety of sentences are labeled for those predicates.

❖The labeling process entails identifying the frame that an instance of the predicate lemma invokes, then identifying semantic arguments for that instance, and tagging them with one of the predetermined set of frame elements for that frame.

❖ The combination of the predicate lemma and the frame that its instance invokes is called a **lexical unit (LU).**

FrameNet:

Key components of FrameNet include:

❖**Frames:** Frames are abstract representations of specific scenarios, events, or conceptual structures. Each frame is associated with a set of frame elements, which are the roles or participants involved in that particular frame.

❖**Lexical Units (LUs):** Lexical Units are words or multi-word expressions that evoke a particular frame. Each LU is linked to a specific frame, and it provides information about how a word is used in context.

❖**Frame Elements (FEs):** Frame Elements are the roles or participants associated with a frame. They represent the semantic roles played by different participants in a given scenario.

❖**Example Sentences:** FrameNet includes annotated example sentences that illustrate how words are used in context within specific frames. These examples help to demonstrate the meaning and usage of words in different situations.

❖**Semantic Role Labeling (SRL):** FrameNet is also used for the task of Semantic Role Labeling, which involves automatically identifying and classifying the semantic roles of words in a sentence.

FrameNet:

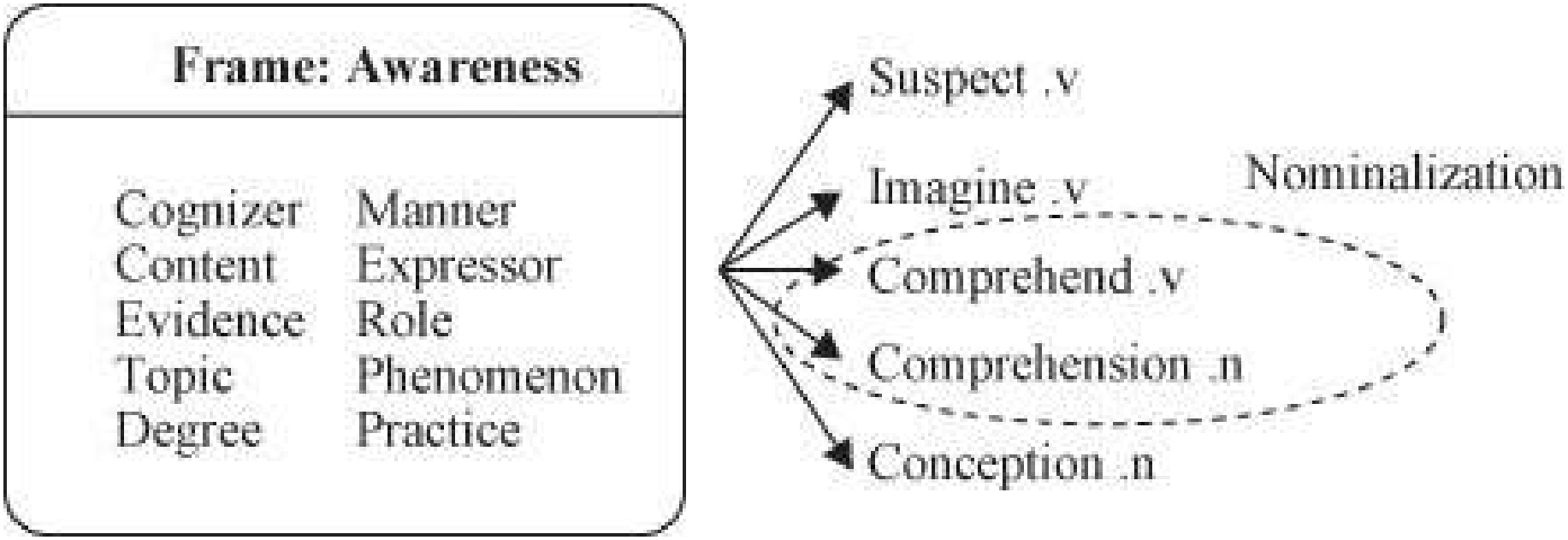
❖Each sense of a polysemous word tends to be associated with a unique frame. For example, the verb

break can mean ***fail to observe (a law, regulation, or agreement)***

and can belong to a COMPLIANCE frame along with other word meanings such as

violation, obey, flout; or it can mean *cause to suddenly separate into pieces in a destructive manner* and can belong to a CAUSE_TO_FRAGMENT frame along with other meanings such as *fracture, fragment, smash*.

FrameNet:



The frame AWARENESS is instantiated by the verb predicate *believe* and the noun predicate *comprehension*. shows the AWARENESS frame along with the frame-elements and a sample set of predicates that involve it including verbs and nominalizations.

FrameNet:

Girish Lectures
Learning from Experience, Sharing with Passion

•**Frame: Awareness** - This frame represents situations where a cognizer becomes aware of something, encompassing actions like suspecting, imagining, and comprehending.
•**Frame Elements** - These are the semantic roles within the frame, including:

1. **Cognizer** - the entity that becomes aware or perceives something.
2. **Content** - the information or concept that is being understood or imagined.
3. **Evidence** - the information that leads the cognizer to awareness.
4. **Topic** - the subject of awareness.
5. **Degree** - the extent of awareness.
6. **Manner** - the way in which awareness is achieved.
7. **Expressor** - the entity expressing awareness.
8. **Role** - the position of the cognizer in the awareness event.
9. **Phenomenon** - the entity or situation that is being perceived.
10. **Practice** - the activity involved in achieving awareness.

•**Lexical Units** - On the right side, verbs like "suspect," "imagine," and "comprehend" are associated with this frame. Some of these verbs can have nominalizations, shown in dashed ovals (e.g., "comprehend" → "comprehension," "concept" → "conception"). Nominalization refers to forming nouns from verbs, like turning "comprehend" into "comprehension."

FrameNet:

Girish Lectures
Learning from Experience, Sharing with Passion

- [$_{\textbf{Cognizer}}$ We] [$_{\textbf{Predicate:verb}}$ *believe*] [$_{\textbf{Content}}$ it is a fair and generous price]

- No doubts existed as to [$_{\textbf{Cognizer}}$ our] [$_{\textbf{Predicate:noun}}$ *comprehension*] [$_{\textbf{Content}}$ of it]

- FrameNet encompasses a wide variety of nominal predicates.
- They include nominals, and nominalizations.

The ***good*** **son** loves the *wild* dog.

Verbs/Adjectives	Nominalizations
Demonstrate	Demonstration
Remove	Removal
Argue	Argument
Receive	Receipt
Susceptible	Susceptibility
Abnormal	Abnormality
Dry	Dryness

- FrameNet also contains some adjective and preposition predicates.

FrameNet:

Frame: Commerce_Buy

Target Word: purchase

Example Sentence: "She purchased a new laptop online.

"Frame Elements:

1. Buyer (she)
2. Goods (laptop)
3. Seller (online retailer)
4. Medium (online)
5. Price (not specified)

Semantic Roles:

1. Agent (she, performing the action of purchasing)
2. Theme (laptop, the thing being purchased)
3. Source (online retailer, the entity providing the goods)

FrameNet:

- **Frame: Motion_Move**
  Target Word: walk
  Example Sentence: "He walked to the store.“

- **Frame: Communication_Speak**
  Target Word: discuss
  Example Sentence: "They discussed the project details.“

- **Frame: Emotion_Happiness**
  Target Word: joyful
  Example Sentence: "She felt joyful on her birthday."

PropBank:

❖The main goal of PropBank is to create a corpus of text annotated with information about who did what to whom, where, when, why, and how, for a set of verbs.
❖This annotation is used to train and evaluate systems that aim to automatically extract information about events and their participants from text.
❖Each verb in PropBank is associated with a set of numbered roles, and examples in the corpus are annotated to indicate which phrases in a sentence fill these roles.

❖For example, for the verb "give," PropBank might include roles such as "Arg0" for the giver, "Arg1" for the thing given, and "Arg2" for the recipient.
❖An example sentence might be annotated as follows:
The professor [Arg0] gave [V] the book [Arg1] to the student [Arg2].

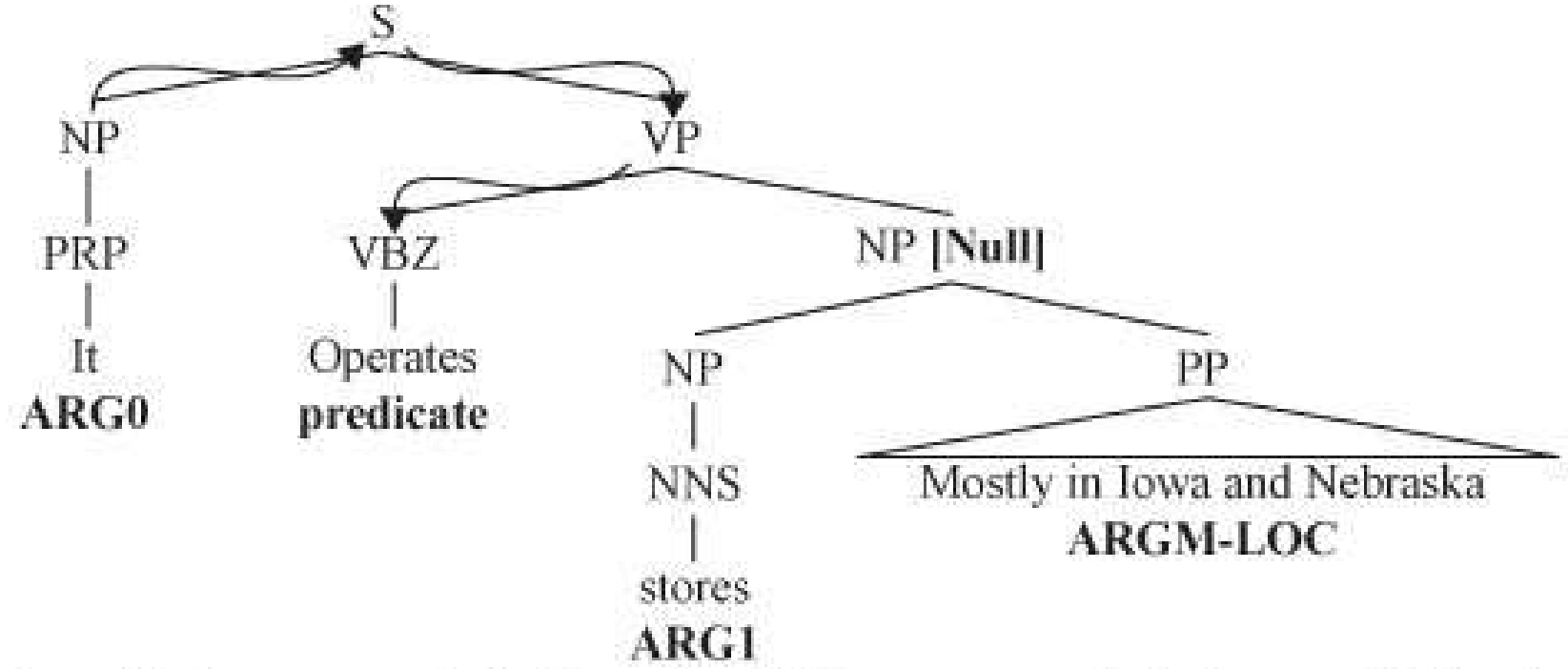
PropBank:

Predicate	Argument	Description
operate.01		
	ARG0	Agent, operator
	ARG1	Thing operated
	ARG2	Explicit patient (thing operated on)
	ARG3	Explicit argument
	ARG4	Explicit instrument
author.01		
	ARG0	Author, agent
	ARG1	Text authored

Tag	Description	Examples
ARGM-LOC	Locative	*the museum, in Westborough, Mass.*
ARGM-TMP	Temporal	*now, by next summer*
ARGM-MNR	Manner	*heavily, clearly, at a rapid rate*
ARGM-DIR	Direction	*to market, to Bangkok*
ARGM-CAU	Cause	*In response to the ruling*
ARGM-DIS	Discourse	*for example, in part, Similarly*
ARGM-EXT	Extent	*at $38.375, 50 points*
ARGM-PRP	Purpose	*to pay for the plant*
ARGM-NEG	Negation	*not, n't*
ARGM-MOD	Modal	*can, might, should, will*
ARGM-REC	Reciprocals	*each other*
ARGM-PRD	Secondary Predication	*to become a teacher*
ARGM	Bare ARGM	*with a police escort*
ARGM-ADV	Adverbials	*(none of the above)*

PropBank:

Syntax tree for a sentence illustrating the PropBank tags



[ARG0 It] [predicate operates] [ARG1 stores] [ARGM-LOC mostly in Iowa and Nebraska].

PropBank:

❖An important distinction to note between the FrameNet and PropBank corpora is that in FrameNet there exist lexical units, which are words paired with their meanings or the frames that they invoke, whereas in PropBank each lemma has a list of different **framesets** that represent all the senses for which there is a different argument structure.

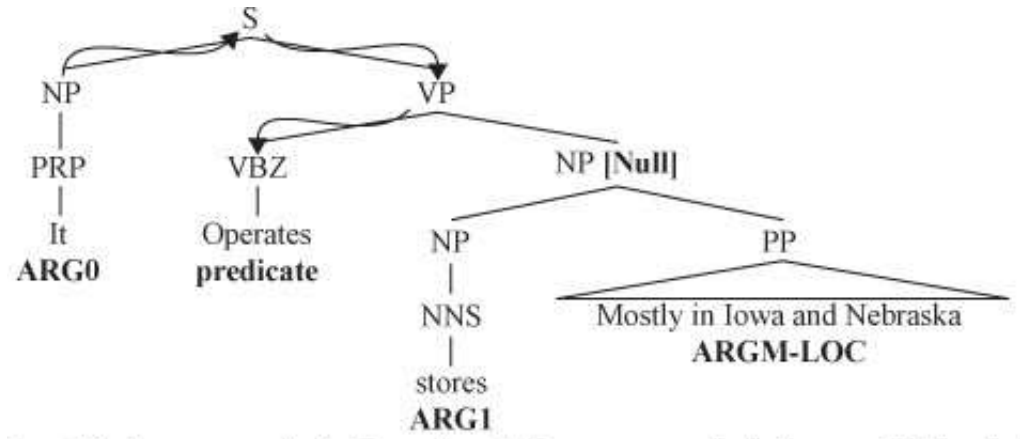
buy.v	PropBank	FrameNet
Frame	buy.01	Commerce_buy
Roles	ARG0: buyer ARG1: thing bought ARG2: seller ARG3: paid ARG4: benefactive ...	Buyer Goods Seller Money Recipient ...

Systems:

❖**Argument identification**—This is the task of identifying all and only the parse constituents that represent valid semantic arguments of a predicate.

❖**Argument classification**—Given constituents known to represent argu- ments of a predicate, assign the appropriate argument labels to them.

❖**Argument identification and classification**—This task is a combination of the previous two tasks, where the constituents that represent arguments of a predicate are identified, and the appropriate argument label is assigned to them.



Syntactic Representations:

Algorithm 4–3. The semantic role labeling (SRL) algorithm:

Procedure: SRL(sentence) **returns** best *semantic role labeling*

Input: *syntactic parse* of the *sentence*

1: *generate* a full syntactic parse of the *sentence*
2: *identify* all the *predicates*
3: **for all** *predicate* ∈ *sentence* **do**
4: *extract* a set of features for each node in the tree relative to the *predicate*
5: *classify* each feature vector using the *model* created in training
6: *select* the class of highest scoring classifier
7: **return** best *semantic role labeling*
8: **end for**

Syntactic Representations:

❖Key aspects of Semantic Role Labeling:

❖**Roles and Arguments:** In a sentence, verbs typically represent actions, and semantic roles are the different participants or entities involved in those actions. Common roles include the agent (the one performing the action), the patient (the entity affected by the action), the theme (the entity undergoing the action), and various others depending on the verb and the context.

❖**Annotation:** Training data for Semantic Role Labeling often involves annotated corpora where sentences are marked with information about the semantic roles of words. The Penn PropBank is a prominent example of such annotated data used for SRL.

❖**Computational Models:** SRL is typically addressed using machine learning and computational models. These models learn from annotated data to predict the semantic roles of words in new, unseen sentences. Features for these models include syntactic information, contextual information, and word embeddings.

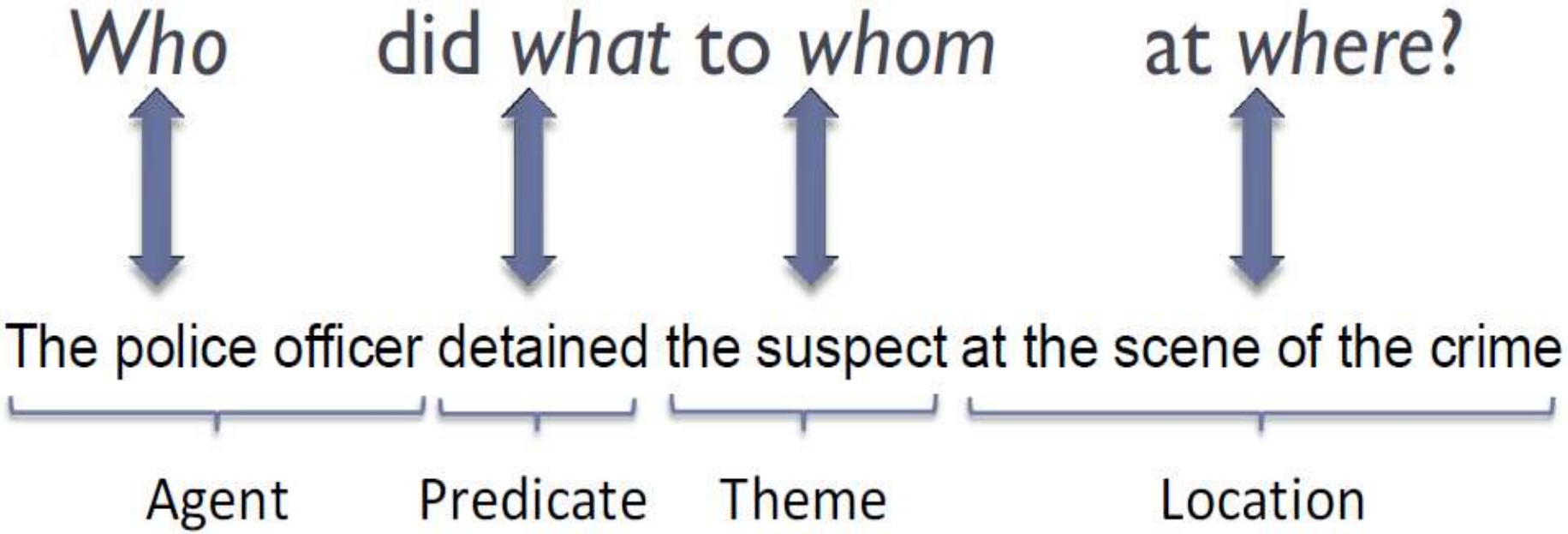
❖**Evaluation:** The performance of SRL systems is evaluated based on their ability to correctly predict the semantic roles in a given sentence. Common evaluation metrics include precision, recall, and F1 score.

❖**Applications:** Semantic Role Labeling has various applications in NLP and information extraction. It is crucial for tasks such as question answering, information retrieval, and building more sophisticated language understanding systems.

❖**FrameNet and PropBank:** FrameNet and PropBank are two resources commonly used for developing and evaluating SRL systems. PropBank focuses on verbs and their roles, while FrameNet provides a broader set of frames representing various scenarios and events in which words are used.

Syntactic Representations:

Semantic Role Labeling



Syntactic Representations:

Can we figure out that these have the same meaning?

XYZ corporation **bought** the stock.

They **sold** the stock to XYZ corporation.

The stock was **bought** by XYZ corporation.

The **purchase** of the stock by XYZ corporation...

The stock **purchase** by XYZ corporation...

Syntactic Representations:

❖**Phrase Structure Grammar (PSG)** FrameNet marks word spans in sentences to represent arguments, whereas PropBank tags nodes in a tree- bank tree with arguments.
❖**Path—**This feature is the syntactic path through the parse tree from the parse constituent to the predicate being classified.
❖**Predicate—**The identity of the predicate lemma is used as a feature.
❖ **Phrase type—**This feature is the syntactic category (NP, PP, S, etc.) of the constituent to be labeled.
❖**Position—**This feature is a binary feature identifying whether the phrase is before or after the predicate.
❖**Voice—**This feature indicates whether the predicate is realized as an active or passive construction.
❖**Head word—**This feature is the syntactic head of the phrase.
❖**Subcategorization—**This feature is the phrase structure rule expanding the predicate's parent node in the parse tree.
❖the subcategorization for the predicate *operates* is VP→VBZ-NP.

Syntactic Representations:

❖**Verb clustering—**The predicate is one of the most salient features in pre- dicting the argument class. Given various syntactic/semantic construc- tions that a predicate can appear in, any amount of hand-tagged training data would be relatively limited for estimating the parameters of a model, and any real-world test set will contain predicate sense/frames that have not been seen in training.

❖For example, verbs such as *eat*, *devour*, and *savor* will tend to all occur with direct objects describing food.

❖The clustering algorithm uses a database of verb–direct object relations extraction.


Thank you!!!...

Girish Lectures

Learning from Experience, Sharing with Passion

Natural Language Processing

Semantic Parsing

Content word

❖**Content word—**Because the head word feature of some constituents, such as PP and SBAR, is not very informative, they defined a set of heuristics for some constituent type.
❖A different set of rules was used to identify a so-called **content** word instead of using the usual head word–finding rules. This was used as an additional feature.

❖Content words are words that carry significant meaning in a sentence, excluding function words. They typically belong to the following parts of speech:

❖1. Nouns (e.g., dog, city, teacher)
❖2. Verbs (e.g., run, eat, write)
❖3. Adjectives (e.g., happy, big, blue)
❖4. Adverbs (e.g., quickly, very, well)

❖Content words are essential for understanding the semantic meaning of a sentence.

Content word

❖Examples:

❖- In the sentence "The happy dog ran quickly," the content words are:
- Happy (adjective)
- Dog (noun)
- Ran (verb)
- Quickly (adverb)

Syntactic Representations:

List of content word heuristics

H1: **if** phrase type is PP **then** select the rightmost child
Example: phrase = "in Texas," content word = "Texas"
H2: **if** phrase type is SBAR **then** select the leftmost sentence (S*) clause
Example: phrase = "that occurred yesterday," content word = "occurred"
H3: **if** phrase type is VP **then**
if there is a VP child **then**
select the leftmost VP child
else
select the head word
Example: phrase = "had placed," content word = "placed"
H4: **if** phrase type is ADVP **then** select the rightmost child, not IN or TO
Example: phrase = "more than," content word = "more"
H5: **if** phrase type is ADJP **then** select the rightmost adjective, verb, noun, or ADJP
Example: phrase = "61 years old," content word = "61"
H6: **for all** other phrase types select the head word
Example: phrase = "red house," content word = "red"

Resource: **Bikel, Daniel; Zitouni, Imed. Multilingual Natural Language Processing Applications: From Theory to Practice (IBM Press) (p. 125). Pearson Education.**

Syntactic Representations:

❖**Part of speech of the head word and content word—**Adding part of speech of the head word and content word of a constituent as a feature to help generalize in the task of argument identification gave a significant performance boost to their decision tree-based system.

❖**Named entity of the content word—**Certain roles, such as ARGM-TMP and ARGM-LOC, tend to contain TIME or PLACE named entities. This information was added as a set of binary-valued features.

❖**Boolean named entity flags—**Surdeanu et al. also added named entity information as a feature. They created indicator functions for each of the seven named entity types: PERSON, PLACE, TIME, DATE, MONEY, PERCENT, ORGANIZATION.

❖**Phrasal verb collocations—**This feature comprises frequency statistics related to the verb and the immediately following preposition.

❖**Logical function—**This is a feature that takes three values—external ar- gument, object argument, and other argument—and is computed using some heuristics on the syntax tree.

Syntactic Representations:

❖**Order of frame elements—**This feature represents the position of a frame element relative to other frame elements in a sentence.

❖**Syntactic pattern—**This feature is also generated using heuristics on the phrase type and the logical function of the constituent.

❖**Previous role**—This is a set of features indicating the ***n*th** previous role that had been observed/assigned by the system for the current predicate.

❖**Named entities in constituents:-** A performance improvement on classifying the semantic role of the constituent by using the presence of a named entity in the constituent.

❖Some of these entities, such as location and time, are particularly important for the ad-junctive arguments ARGM-LOC and ARGM-TMP.

❖Entity tags should also help in cases where the head words are not common or for a closed set of loca- tive or temporal cues, such as *in Mexico*, or *in 2003*.

Syntactic Representations:

❖**Verb sense information—**The arguments that a predicate can take depend on the sense of the predicate.

❖ Each predicate tagged in the PropBank corpus is assigned a separate set of arguments depending on the sense in which it is used. This is also known as the **frameset ID**.

❖Depending on the sense of the predicate *talk*, either ARG1 or ARG2 can identify the *hearer*. Absence of this information can be potentially confusing to the learning mechanism.

❖**Noun head of prepositional phrases—**Many adjunctive arguments, such as temporals and locatives, occur as prepositional phrases in a sentence, and the head words of those phrases, which are always prepositions, of- ten are not very discriminative.

❖For instance, *in the city* and *in a few minutes* both share the same head word *in*, and neither contains a named entity, but the former is ARGM-LOC, whereas the latter is ARGM-TMP.

Syntactic Representations:

❖**Verb sense information—**The arguments that a predicate can take depend on the sense of the predicate.

❖ Each predicate tagged in the PropBank corpus is assigned a separate set of arguments depending on the sense in which it is used. This is also known as the **frameset ID**.

❖Depending on the sense of the predicate *talk*, either ARG1 or ARG2 can identify the *hearer*. Absence of this information can be potentially confusing to the learning mechanism.

❖**Noun head of prepositional phrases—**Many adjunctive arguments, such as temporals and locatives, occur as prepositional phrases in a sentence, and the head words of those phrases, which are always prepositions, of- ten are not very discriminative.

❖For instance, *in the city* and *in a few minutes* both share the same head word *in*, and neither contains a named entity, but the former is ARGM-LOC, whereas the latter is ARGM-TMP.

❖Replaced the head word of a prepositional phrase with that of the first noun phrase inside the prepositional phrase.

❖The preposition information was retained by appending it to the phrase type; for example, the head word of the prepositional phrase *for about 20 minutes* was originally the preposition *for*.

Syntactic Representations:

❖**First and last word/POS in constituent—**Some arguments tend to con- tain discriminative first and last words, so these were used along with their part of speech as four new features.

❖ **Ordinal constituent position—**This feature avoids false positives where constituents far away from the predicate are spuriously identified as arguments. It is a concatenation of the constituent type and its ordinal position from the predicate.

❖**Constituent tree distance—**This is a finer way of specifying the already present position feature, where the distance of the constituent from the predicate is measured in terms of the number of nodes that need to be traversed through the syntax tree to go from one to the other.

❖**Constituent relative features—**These are nine features representing the constituent type, head word and head word part of speech of the parent, and left and right siblings of the constituent in focus.

❖These were added on the intuition that encoding the tree context this way might add robust-ness and improve generalization.

Syntactic Representations:

❖**Temporal cue words—**Several temporal cue words are not captured by the named entity tagger and were therefore added as binary features indicating their presence.

❖ The BOW (Bag of words) toolkit was used to identify words and bigrams that had highest average mutual information with the ARGM-TMP argument class.

❖**Dynamic class context—**In the task of argument classification, these are dynamic features that represent the hypotheses of, at most, the previous two non-null nodes belonging to the same tree as the node being classified.

❖**Path -T**he path is one of the most salient features. However, it is also the most datasparse feature. To overcome this problem, the path was generalized in several different ways:

Clause-based path variations—Position of the clause node (S, SBAR) seems to be an important feature in argument identification .

Path *n*-grams—This feature decomposes a path into a series of trigrams. For example, the path NP↑S↑VP↑SBAR↑NP↑VP↓VBD becomes NP↑S↑VP, S↑VP↑SBAR, VP↑SBAR↑NP, SBAR↑NP↑VP, and so on. Shorter paths were padded with nulls.

Syntactic Representations:

Single-character phrase tags—Each phrase category is clustered to a cat- egory defined by the first character of the phrase label.

Path compression—Compressing sequences of identical labels into fol- lowing the intuition that successive embedding of the same phrase in the tree might not add additional information.

Directionless path—Removing the direction in the path, thus making in- significant the point at which it changes direction in the tree.

Partial path—Using only that part of the path from the constituent to the lowest common ancestor of the predicate and the constituent. For example, the partial path for the path illustrated in parse tree is NP↑S.

Syntactic Representations:

Predicate context—This feature captures predicate sense variations. Two words before and two words after were added as features. The part of speech of the words were also added as features.

Punctuations—For some adjunctive arguments, punctuation plays an im- portant role. This set of features captures whether punctuation appears immediately before and after the constituent.

❖**Feature context—**Features of constituents that are parent or siblings of the constituent being classified were found useful.

❖Traditionally, each constituent is classified independently; however, in reality, there is a complex interaction between the types and number of arguments that a constituent can have. In other words, the classification of each argument is dependent on the classifications of other nodes.

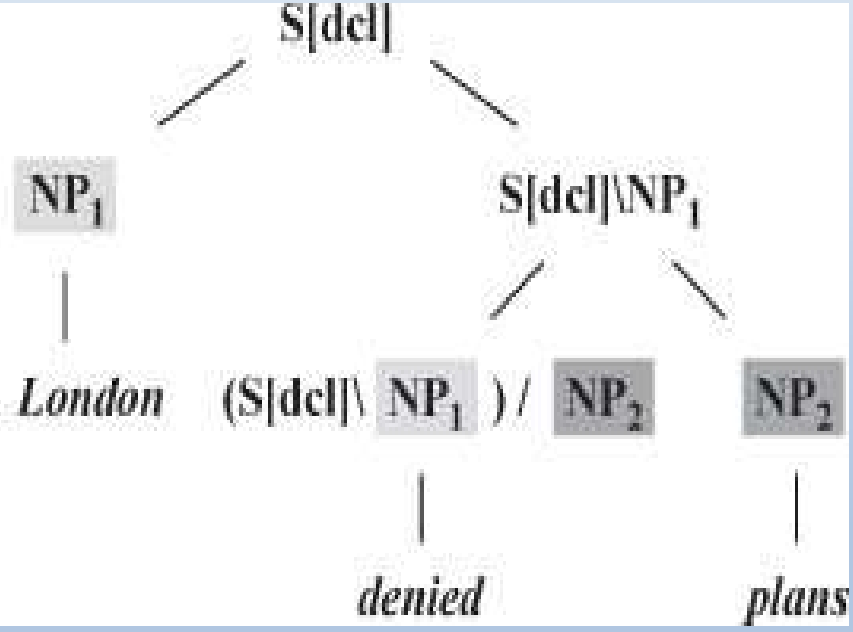
Syntactic Representations:

Girish Lectures
Learning from Experience, Sharing with Passion

❖**Combinatory Categorial Grammar (CCG):**

London denied plans on Monday

w_h	w_a	c_h	i
denied	*London*	$(S[dcl]\backslash NP_1)/NP_2$	1
denied	*plans*	$(S[dcl]\backslash NP_1/NP_2$	2
on	*denied*	$((S\backslash NP_1\backslash (S\backslash NP)_2/NP)_3$	2
on	*Monday*	$((S\backslash NP_1\backslash (S\backslash NP)_2/NP)_3$	3



Syntactic Representations:

Combinatory Categorial Grammar (CCG):

❖Combinatory Categorial Grammar (CCG) is a formal linguistic framework that falls within the broader category of categorial grammars.

❖ It is a type of generative grammar that seeks to describe the syntax of natural language by using a combination of function application and type-raising operations.

❖CCG representation improves semantic role label- ing performance on core arguments .

❖Because CCG trees are binary trees and the constituents have poor alignment with the semantic arguments of a predicate, the researchers performed experiments using head words rather than the entire span.

❖**Phrase type—**This is the category of the maximal projection between the two words, the predicate and the dependent word.

Syntactic Representations:

❖**Categorial path**—This is a feature formed by concatenating the following three values: (i) category to which the dependent word belongs, (ii) the direction of dependence, and (iii) the slot in the category filled by the dependent word.

❖**Tree Path**—This is the categorial analogue of the path feature in the Charniak parsebased system, which traces the path from the dependent word to the predicate through the binary CCG tree.

❖**Tree-Adjoining Grammar (TAG):**

- ❖ Two different sets of features:
- ❖ (i) surface syntactic features and (ii) additional features that result from the extraction of a TAG from the Penn Treebank.

❖**Supertag path**—This feature is the same as the path feature seen earlier except that in this case it is derived from a TAG rather than from a PSG.

❖ **Supertag**—This feature is the tree frame corresponding to the predicate or the argument.

❖ **Surface syntactic role**—This feature is the surface syntactic role of the argument.

❖**Surface subcategorization**—This feature is the subcategorization frame.

Syntactic Representations:

Deep syntactic role—This feature is the deep syntactic role of an argu- ment, whose values include *subject* and *direct object*.

Deep subcategorization—This is the deep syntactic sub categorization frame.

For example, for a transitive verb, it would be NP0 NP1. When available, the preposition modifying the NP is used to lexicalize the feature. So, in case of the predicate *load*, a possible frame would be NP0 NP1 NP2(into).

Systems:

❖**Disallowing Overlaps** Since each constituent is classified independently of the other, it is possible that two constituents that overlap get assigned an argument type.

❖Because we are dealing with parse tree, nodes overlap- ping in words always have an ancestor-descendant relationship.

❖**Example 4–1:** But [$_{ARG0}$ nobody] [$_{predicate}$ knows] [$_{ARG1}$ at what level [$_{ARG1}$ the futures and stocks will open today]]

❖This is a problem because overlapping arguments were not allowed in PropBank (or, more specifically, any two arguments of a verb predicate even in FrameNet cannot overlap).

Girish Lectures

Learning from Experience, Sharing with Passion

Meaning Representation

Meaning Representation:

❖Meaning representation is the process of converting natural language input into a format that is unambiguous and easily understandable by a machine or end application, which can take actions based on the input.

❖So far, there is no one universally accepted representation, so these systems and their representations tend to be domain specific.

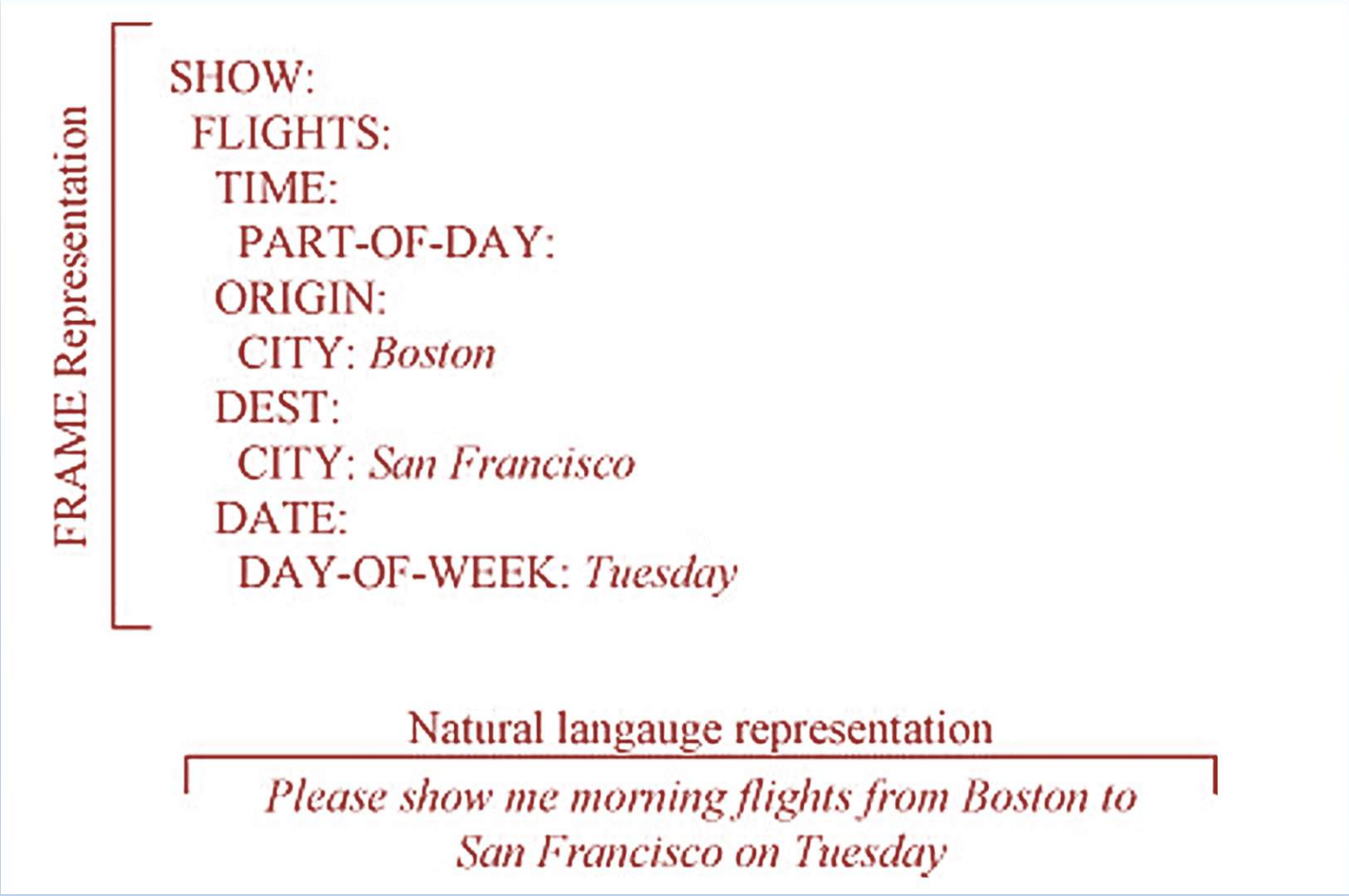
❖**ATIS:**Air Travel Information System (ATIS) project is considered one of the first concerted efforts to build systems to transform natural language into a representation that could be used by an end application to make decisions.

❖The task involved a machine to transform a user query in spontaneous speech, using a restricted vocabulary, about flight information.

❖ It then formed a representation that was compiled into a SQL query, which was used to extract answers from a flight database.

Meaning Representation:

A sample user query and its frame representation in the ATIS program



Meaning Representation:

A sample user query and its frame representation in the ATIS program

ATIS is a spoken language database containing audio recordings and transcripts of people querying a travel information system. The dataset aims to evaluate speech recognition, natural language understanding, and dialogue management systems.

Key Features

1. **Domain:** Air travel information
2. **Tasks:** Flight information, booking, and scheduling
3. **Data types:** Audio recordings, transcripts, and semantic labels
4. **Size:** Approximately 5,000 audio recordings and 14,000 transcripts
5. **Language:** English

Meaning Representation:

❖**Communicator**: The Communicator program was the follow-on to ATIS. While ATIS was more focused on **user-initiated dialog** (i.e., the user asked questions to the machine, which provided answers), Communicator involved a **mixed- initiative dialog**, whereby the human and machine had a dialog with each other with the computer presenting users with real-time travel information and helping them negotiate a preferred itinerary.

❖**GeoQuery:** In the domain of U.S. geography, there is a natural language interface (NLI) to a geographic database called Geobase, which has about 800 Prolog facts stored in a relational database with geographic information such as population, neighboring states, major rivers, and major cities.

Some sample queries and their representations are as follows:

1. What is the capital of the state with the largest population?

answer(C, (capital(S, C), largest(P, (state(S), population(S, P))))

2. What are the major cities in Kansas?

answer(C, (major(C), city(C), loc(C, S), equal(S, stateid(kansas))))

Meaning Representation:

❖**Robocup: Clang**

RoboCup (**www.robocup.org**) is an international initiative by the artifi- cial intelligence community that uses robotic soccer as its domain. There is a special formal language, CLang, which is used to encode the advice from the team coach, and the behaviors are expressed as if-then rules.

Following is an example representation in this domain:

1. If the ball is in our penalty area, all our players except player 4 should stay in our half.

((bpos (penalty-area our)) (do (player-except our 4) (pos (half our))))

Meaning Representation:

Systems:

Meaning representation can be a SQL query, a Prolog query, or a domain-specific query representation.

❖ Now we look at the various ways the problem of mapping the natural language to such meaning representation has been tackled.

❖**Rule Based:** The ATIS and Communicator projects were rule-based systems in the sense that they used an interpreter whose semantic grammar was handcrafted to be robust to speech recognition errors.

❖**Supervised:** Rule-based techniques are relatively easy to craft in the beginning and serve a good purpose to formulate solutions to various tasks, they have several downsides:

(i) they need some effort upfront to create the rules,

(ii) the time and specificity required to write rules usually re- stricts the development to systems that operate in limited domains,

(iii) they are hard to maintain and scale up as the problem becomes more complex and more domain independent, and

(iv) they tend to be brittle.

Girish Lectures

Learning from Experience, Sharing with Passion